OneMoreTest: A Learning-Based Approach to Generating and Selecting Fault-Revealing Unit Tests

Wei Wei, Yanjie Jiang, Yahui Li, Lu Zhang, and Hui Liu

***Abstract*—Developers often manually design a few unit tests for a given method under development. After passing such manually designed tests, however, they usually have to turn to automated test case generation tools like EvoSuite and Randoop for more thorough testing. Although the automatically generated tests may achieve a high coverage, they rarely identify hard-to-detect defects automatically because of the well-known test oracle problem: It is challenging to tell whether the output is correct or incorrect without explicit test oracle (expected output). Consequently, developers should manually select and verify a few suspicious test cases to identify hard-to-detect defects. To this end, in this paper, we propose a novel approach, called *OneMoreTest*, to generating and selecting the most suspicious tests for manual verification. Based on a manually designed passed test, *OneMoreTest* automatically generates millions of input-output pairs for the method under test (MUT) with mutation-based fuzzing. It then trains an automatically generated neural network to simulate the MUT's behavior. For new tests automatically generated for the same MUT, *OneMoreTest* suggests developers with the top k most suspicious tests that have the greatest distances between their actual output and estimated output (i.e., network's output). Our evaluation on real-world faulty methods suggests that *OneMoreTest* is accurate. On 70.79% of the involved 178 real-world faulty methods, we can identify the defects by manually verifying only a SINGLE test for each of the methods according to *OneMoreTest*'s suggestions. Compared against the state of the art, *OneMoreTest* improved the precision from 46.63% to 72.62%, and recall from 46.63% to 70.79%.**

***Index Terms*—Unit testing, test oracle, deep learning, test case selection.**

I. Introduction

UNIT testing is widely and successfully employed to improve software quality [1]. It is likely that developers would manually design a small number of tests for each non-trivial method they develop [2], and would validate the method against such tests. Such manually designed tests are highly valuable because they often either represent the major scenarios of the method under test (called MUT for short) or represent the major risk in method implementation [3]. However, the number of manually designed tests is often small because manual design of tests is tedious and time-consuming. Consequently, passing such tests alone is often insufficient to guarantee MUT's quality.

To this end, after passing the manual design tests, developers have to employ automated testing tools like EvoSuite [4], Randoop [5], and TestSuite [6] to generate and run a large number of tests. Such tools have the potential to conduct a thorough test automatically, achieving a high test coverage. However, it remains challenging to automatically verify the generated tests, i.e., whether and which tests reveal defects. In general, we should verify the tests by comparing the actual output of the MUT against the expected output. A test passes only if the actual output is equivalent to the corresponding expected output. However, the expected output is often not available. To the best of our knowledge, most test case generation tools cannot generate the expected output of the MUT. It is also challenging and time-consuming to specify the expected output manually, especially when the number of tests is large. The challenges in automated verification of tests are known as the *test oracle problem* [7], which is deemed as the bottleneck to test automation. Lacking test oracles, it is challenging to automatically identify hard-to-detect defects in software, and most of the automatically generated tests are discarded unless they result in general and well-known errors like crashes.

To improve the confidence in unit tests and to increase the chance of detecting hard-to-detect defects, in this paper, we propose a novel approach (called *OneMoreTest*) to generating and selecting fault-revealing unit tests. It can be formulated as an optimization problem to generate various test cases for a given method according to an existing test case, and to select the most promising one from the generated test cases, maximizing the likelihood of triggering the buggy behavior of the specified method. In this work, we focus on method-level unit tests, aiming to capture the behavioral characteristics of individual methods. By targeting this granularity level, we can better identify and prioritize fault-revealing tests for specific methods.

Based on a manually designed unit test, *OneMoreTest* automatically generates millions of input-output pairs for the MUT, trains a neural network to simulate the method's behavior, and

Received 20 September 2024; revised 10 June 2025; accepted 13 June 2025. Date of publication 25 June 2025; date of current version 15 August 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62232003 and Grant 62172037. Recommended for acceptance by M. Monperrus. *(Corresponding author: Hui Liu.)*

Wei Wei, Yahui Li, and Hui Liu are with the School of Computer Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: 1436784631@qq.com; yahui1178@gmail.com; liuhui08@bit.edu.cn).

Yanjie Jiang is with the College of Intelligence and Computing, Tianjin University, Tianjin 300072, China, and also with the Key Laboratory of High Confidence Software Technologies, Ministry of Education, Peking University, Beijing 100871, China (e-mail: 2990094974@qq.com).

Lu Zhang is with the School of Computer Science, Peking University, Beijing 100871, China (e-mail: zhanglucs@pku.edu.cn).

Digital Object Identifier 10.1109/TSE.2025.3581556

0098-5589 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

leverages the resulting network to select the most suspicious tests for manual inspection. It first automatically mutates the manually designed tests with mutation-based fuzzing, replacing hard-coded input with random values. These tests are run multiple times, collecting numerous input-output pairs for the MUT. It also automatically generates a neural network according to the input-output pairs. The automatically collected input-output pairs are then used to train the network that should learn how to estimate the output of the MUT according to the given input. *OneMoreTest* executes the mutated tests again and thus generates an additional set of input-output pairs. *OneMoreTest* tests them on the trained neural network, selects the top item with the greatest loss, and presents the associated test to developers for manual validation. The rationale is that the network trained with numerous tests should fit well the majority of the tests whereas outliers are less fitted (and thus have greater loss). Existing studies [8] suggest that neural networks have the potential to simulate software entities such as methods. However, they often struggle to capture edge cases that are difficult to trigger, resulting in networks that may behave differently from the original software entity in these scenarios. By employing differential testing, which involves comparing the network's output with that of the original method, we can identify such edge cases. These cases, compared to mainstream ones, are more likely to reveal hard-to-detect defects. For datasets like Defects4J, edge cases can be further validated by comparing the outputs of the buggy and fixed versions of a method to determine their correctness. For general projects without predefined fixed versions, OneMoreTest identifies and prioritizes edge cases by ranking them based on their prediction loss and presents the top-ranked cases to developers for manual inspection. If no incorrect outputs are found among the top k test cases (e.g., top 10), the MUT can likely be deemed safe. This approach effectively reduces the manual effort required by pre-filtering and focusing on the most suspicious test cases.

We evaluated *OneMoreTest* on 178 real-world faulty methods from the widely used Defects4J (version 1.0.0) [9]. Our evaluation results suggest that according to its ranking, on 130 out of the 178 methods, we can identify the defects by manually verifying only a single test for each of the faulty methods. Our evaluation results also suggest that *OneMoreTest* is more effective than traditional test case selection approaches in suggesting the most suspicious unit tests. Compared against the state of the art, *OneMoreTest* improved the precision from 46.63% to 72.62%, and recall from 46.63% to 70.79%.

The paper makes the following contributions:

- A novel approach to generating and selecting fault-revealing unit tests.
- A prototype implementation of the approach that is publicly available for further evaluation and further improvement.
- An initial evaluation of the proposed approach on real-world faulty methods whose results suggest that the proposed approach is accurate.

The replication package of the evaluation is publicly available [10], providing the implementation of the proposed approach, the data set for evaluation, and the instructions for replicating the experiment.

The rest of the paper is structured as follows. Section II presents the proposed approach. Section III presents its evaluation. Section IV discusses related issues whereas Section V discusses related work. Section VI makes conclusions.

II. Approach

A. Overview

In this section, we present a learning-based approach (*OneMoreTest*) to generating and selecting fault-revealing unit tests to identify hard-to-detect defects. Fig. 1 presents its overview. For a given MUT and a passed test case of it, *OneMoreTest* works as follows.

- First, *OneMoreTest* replaces the literals in the given test case method invocations with randomly generated values.
- Second, *OneMoreTest* runs the mutated test case automatically for N times, which results in N pairs of input-output (noted as *TrainingData*).
- Third, according to input-output pairs generated in the preceding step, *OneMoreTest* automatically generates an initial neural network based on predefined template. The network should take *input* in *TrainingData* as input, and generates *output* in *TrainingData* as output.
- Fourth, *OneMoreTest* trains the generated neural network with *TrainingData*. If the network is not stable or not accurate, *OneMoreTest* terminates. Otherwise, it goes to the next step.
- Fifth, *OneMoreTest* runs the mutated test case automatically again for M times, which results in M pairs of input-output (noted as *TestingData*).
- Finally, *OneMoreTest* runs the trained network with items in *TestingData*, and selects top k items with the greatest loss. Test cases associated with the selected items are presented to developers for manual validation.

Details of the key components are presented in the following sections.

B. Mutating Test Cases

OneMoreTest takes a passed test as input and mutates it so that the input of the MUT could be randomly generated on each run, and the output (as well as input) could be recorded automatically. For a given test, *OneMoreTest* identifies all JUnit assertions within the test. For each assertion, it locates the actual output used by this assertion. All such actual output together is taken as the output of the MUT. To identify and mutate the input, *OneMoreTest* searches for all literals within the given test case associated with a specific method, and replaces each literal with an invocation of a pre-defined method that can automatically and randomly generate a value (literal) at a time. We detect and replace the hard-coded values as follows:

- First, we employ JavaParser [11], a sate-of-the-art static analysis tool, to parse the test methods and MUT, resulting in abstract syntax trees (ASTs). By traversing the ASTs, we identify all literal nodes, including literals of *numeric*, *boolean*, *string*, and *enum* [12].

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

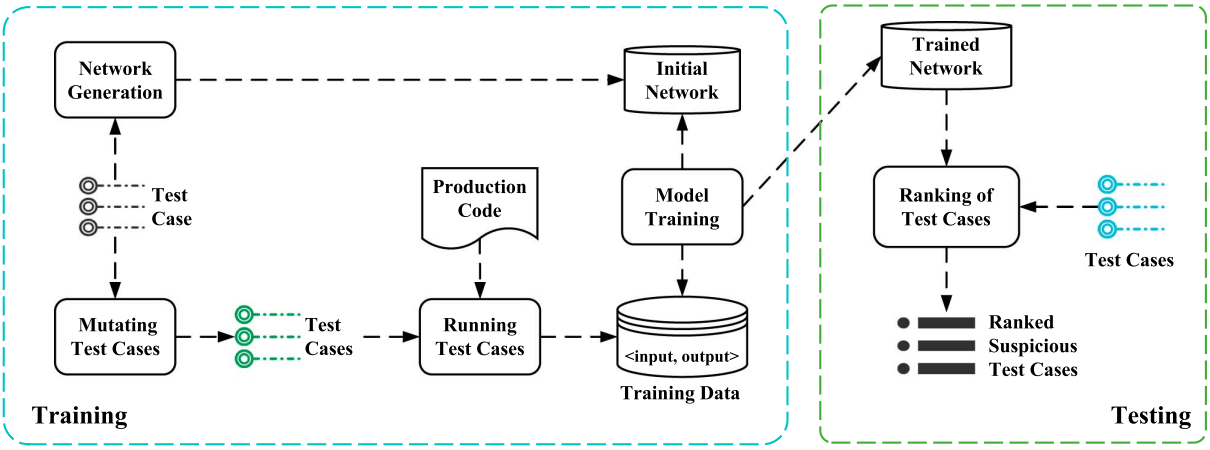

Fig. 1. Overview of *OneMoreTest*.

- Second, we employ a sequence of heuristic rules to infer the range of values that could be used to replace the identified literals. For enumerated types, we extract all the enumerated constants from their definitions to establish the valid range; For array indices, the values must fall between 0 and array.length-1; For each string literal, we request ChatGPT-4o [13] to infer a regular expression to explicitly specify possible replacement values. For other literals, possible replacements should fall into the ranges defined by their corresponding date types.
- Third, we replace the hard-coded literals with pre-defined methods that generate random values. For each type of literal, we manually create a method that generates a random value of the corresponding data type. The method's input parameter is the range inferred in the second step, and its output is a random value within that range. As a result, we replace the hard-coded literals in the original test cases with calls to such methods, enabling the test cases to generate different inputs with each execution. To streamline the use of these pre-defined methods, we package their implementations into a JAR file named *DataGenerator*.

All such generated literals on a single run of the test case are taken as the input of the MUT. *OneMoreTest* also automatically inserts pre-defined statements (that call manually designed logging APIs) to record the identified input and output. Note that such pre-defined methods and APIs are generic and applicable to all JUnit tests. Consequently, they could be defined once and for all. However, non-literal inputs, such as constants defined outside the method and default values of method parameters, are not addressed by the proposed approach and would be ignored.

In practice, we often do not have prior knowledge of existing faults or fault-revealing test cases. Therefore, our approach focuses on generating and prioritizing test cases without relying on predefined assertions. We achieve this by transforming existing test cases to replace constant values with variables generated through randomized functions, and by removing the original assertions to prevent interruptions during the test case generation process. The test case in Listing 1 verifies that the `getN()` method of the `Combinations` class returns the expected value `n` when initialized with specific values of `n` and `k`. Our tool transforms this test case into a form suitable for large-scale test case generation in Listing 2. In the transformed version:

```
@Test
public void testAccessor1() {
    final int n = 5;
    final int k = 3;
    Assert.assertEquals(n, new Combinations(n, k).getN());
}
```

Listing 1. Example Test Case for Direct Return Values

```
@Test
public void testAccessor1() {
    final int n = 5;
    final int k = 3;
    Assert.assertEquals(n, new Combinations(n, k).getN());
}

public void testAccessor1ForGeneration() throws Exception {
    final int n = DataGenerator.generateInteger(minValue1, maxValue1);
    final int k = DataGenerator.generateInteger(minValue2, maxValue2);

    int result = new Combinations(n, k).getN();
    DataGenerator.getOutput(result);
}

@Test
public void testData() {
    try {
        int iteration = 1100000;
        DataGenerator.init();
        for (int i = 0; i < iteration; i++) {
            // Invoke the test method
            testAccessor1ForGeneration();
            DataGenerator.finishTestCase();
        }
    } catch (Exception exception) {
        exception.printStackTrace();
    } finally {
        DataGenerator.close();
    }
}
```

Listing 2. Mutated Test Case for Direct Return Values

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

```
1  public void testEncoderExceptionInit() {
2      String MSG = ‘‘TEST’’;
3      EncoderException e = new EncoderException(MSG);
4      Throwable cause = new Throwable();
5      e.initCause(cause);
6      assertEquals(MSG, e.getMessage());
7      assertNotNull(e.getCause());
8  }
```

Listing 3. Example Test Case for Object State Change

```
1  public void testEncoderExceptionInitForGeneration() {
2      String MSG = DataGenerator.generateString(“[A-F]{1,8}”);
3      EncoderException e = new EncoderException(MSG);
4      Throwable cause = new Throwable();
5      e.initCause(cause);
6      DataGenerator.getOutput(e.getMessage());
7      DataGenerator.getOutput(e.getCause());
8  }
```

Listing 4. Mutated Test Case for Object State Change

- **Randomized Input Generation**: The constants `n` and `k` are replaced with randomly generated integers within specified ranges $[minValue1, maxValue1]$ and $[minValue2, maxValue2]$, respectively. These ranges are determined through static analysis of the method under test (MUT) to ensure valid and meaningful inputs.
- **Output Collection**: Our tool automatically performs a variable extraction operation by invoking the refactor interface in the IDE, specifically the “extract variable” operation. This transforms the code `Assert.assertEquals(n, new Combinations(n, k).getN());` by extracting the expression `new Combinations(n, k).getN()` into a variable, resulting in `int result = new Combinations(n, k).getN();`. Next, the output from the method call `getN()` is captured using `DataGenerator.getOutput(result)`. This step collects outputs for a large number of generated test cases without performing immediate assertions.
- **Test Case Iteration**: The `testData()` method orchestrates the generation of 1,100,000 test cases by repeatedly invoking `testAccessor1ForGeneration()`. This bulk generation is efficient for training our neural network model.
- **Removal of Original Assertions**: The original assertion `Assert.assertEquals(n, new Combinations(n, k).getN());` is removed to prevent assertion failures during generation, which could halt the process.

The test outputs used in our approach are derived from the actual outputs of assertions within the test cases. This ensures that any complex method call chains and their effects are captured naturally in the assertion outputs. Specifically, we evaluate methods that return direct values, modify object states, handle expected exceptions, and so on. Listing 1 and Listing 2 present an example of a test case focusing on direct return values. Listing 3 and Listing 4 demonstrate a test case involving object state changes, while Listing 5 and Listing 6 illustrate a test case designed to handle expected exceptions.

```
1  public void testParseMutableDateTime_simple() {
2      MutableDateTime expect = null;
3      expect = new MutableDateTime(2004, 6, 9, 11, 20, 30, 0, LONDON);
4      assertEquals(expect, g.parseMutableDateTime(‘‘20−06−09T10:20:30Z’’));
5
6      try {
7          g.parseMutableDateTime(‘‘ABC’’);
8          fail();
9      } catch (IllegalArgumentException ex) {}
10 }
```

Listing 5. Example Test Case for Expected Exceptions

```
1  public void testParseMutableDateTime_simpleForGeneration() throws
       Exception {
2      MutableDateTime expect = null;
3      expect = new MutableDateTime(2004, 6, 9, 11, 20, 30, 0, LONDON);
4      MutableDateTime result = g.parseMutableDateTime(DataGenerator.
           generateString(‘‘\\d{2}−\\d{2}−\\d{2}T\\d{2}:\\d{2}:\\d{2}Z’’));
5      DataGenerator.getOutput(result);
6
7      try {
8          g.parseMutableDateTime(DataGenerator.generateString(‘‘[A−Z]{1,10}’’));
9          fail();
10     } catch (IllegalArgumentException ex) {
11         DataGenerator.getOutput(ex.getMessage());
12     }
13 }
```

Listing 6. Mutated Test Case for Expected Exceptions

It should be noted that the output collected by *OneMoreTest* is different from the return value of the MUT. For the test case *testEncoderExceptionInit* in Listing 3, the MUT (i.e., *initCause* on Line 5) does not return anything. However, it does change the state of the object (i.e., *e* defined on Line 3). *OneMoreTest* notates such changes with the actual output of the assertions, i.e., *e.getMessage ()* (Line 6) and *e.getCause ()* (Line 7). Listing 4 shows the mutated version of Listing 3, the string literal “TEST” is replaced with a random string generated by DataGenerator.generateString(“[A-F]{0,8}”), producing a random sequence of 0 to 8 characters from A to F. The outputs, i.e., *e.getMessage()* and *e.getCause()*, are recorded by *DataGenerator.getOutput()* in the output file where different outputs are separated by delimiter “$”. In this case, the output would resemble “[e.getMessage()]$[e.getCause()]”. If a test case contains multiple assert statements, the inputs and outputs for each assert are concatenated in the respective input and output files. For methods that handle expected exceptions, Listing 5 and Listing 6 illustrate a representative case. In Listing 5, the test verifies both the return value and exception handling behavior of the method. Listing 6 demonstrates the mutated version of this test case, where OneMoreTest records outputs related to the thrown exception for further analysis. Of the bugs we tested, 61 out of 178 (34.3%) were caused by exceptions, while 117 out of 178 (65.7%) were logic bugs.

C. Running Test Cases

Once the given test has been mutated successfully, *OneMoreTest* runs the mutated test for N times where N depends on how many training items we need (empirically set to one million).

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

Each run of the mutated test should result in an item:

$$Item_i =< ipt_i, act_i > \quad (1)$$

where ipt_i is the automatically recorded input and act_i is the recorded output. All such items together form the data set for neural network training in the following section.

D. Network Generation and Training

To feed the data item $Item_i$ in Equation 1 into the neural network, we turn the input and output into digital vectors. To this end, non-digital values (strings and characters) are transformed into digital representation by representing each character with its corresponding ASCII and concatenating all such digital ASCIIs as a digital vector. Arrays are decomposed into sequences of primitive elements, each represented as a digital vector. In cases where the MUT returns a complex object or a variable-length list, we handle the output by decomposing the object/list into its constituent primitive attributes and concatenating their individual representations. For instance, if a method returns a list or array of objects, we represent each object independently as a vector and concatenate the resulting vectors to form the final representation of the array. If the size of the list varies, the length of the resulting vector is dynamically determined by the actual size of the list. Because different input has significantly different ranges, we normalize the digital values into the same range [0,1] with min-max normalization.

As a result of the processing specified in the preceding paragraph, we finally represent the input and output in Equation 1 as follows:

$$ipt_i = < dip_{i,1}, \ldots, dip_{i,k} > \quad (2)$$

$$act_i = < dop_{i,1}, \ldots, dop_{i,j} > \quad (3)$$

where $dip_{i,l}$ ($l \in [1, k]$) represents a digital vector for an input, and $dop_{i,l}$ (($l \in [1, j]$)) represents a vector for an output. Both of them range between zero and one. According to the data item, *OneMoreTest* generates neural networks automatically. To improve the generality of the employed deep neural network, *OneMoreTest* only leverages the basic Fully Connected Network where each neuron in a layer is connected to all neurons in the next layer. Notably, the neural network is generated automatically with a predefined template. The template defines the structure of a fully connected neural network that includes one input layer, three hidden layers, and one output layer. The template is instantiated based on the vectors generated from the test cases. The choice of three hidden layers was initially chosen based on empirical knowledge and prior works [14]. Through initial testing on a validation set, we identified the best configuration, which became the default: three hidden layers, each containing 256 neurons. The number of neurons in the input layer corresponds to the maximum length of the input vectors, while the number of neurons in the output layer corresponds to the maximum length of the output vectors.

To evaluate the trade-offs between training data size and model performance, we conducted experiments with total dataset sizes of 11,000, 110,000, 220,000, 550,000, 770,000, and 1,100,000 samples. For each dataset size, samples were split into training, validation, and testing subsets using a fixed ratio of 72.7%, 18.2%, and 9.1%, respectively. The details of the dataset split are summarized in Table I. The experimental results for total dataset sizes ranging from 11,000 to 770,000 are presented in Table IV, where we report precision and recall values for different top@k (k = 1, 10, 100) predictions. For the largest dataset size of 1,100,000 samples, a detailed comparison with other baselines is shown in Table III. The coverage results across methods indicate that the average coverage significantly improves from the original test case to nearly 99% with 10,000 test cases. Beyond this point, as the dataset size increases to 1,000,000 test cases, the coverage stabilizes with limited additional improvements. This demonstrates that smaller datasets are already highly effective in achieving near-optimal coverage while highlighting the diminishing returns of larger datasets. Details of the coverage, as well as related information, are available online [15]. In our approach, test inputs are generated by replacing mutable literals in existing test cases with randomized values within identified ranges. If no specific range is found, we default to the full range of basic data types (e.g., int values from *Integer.MIN_VALUE* to *Integer.MAX_VALUE*).

TABLE I
TRAINING, VALIDATION, AND TEST DATASET SPLITS WITH CONVERGENCE RESULTS

Training Samples	Testing Samples	Validation Samples	Total Samples
8,000	1,000	2,000	11,000
80,000	10,000	20,000	110,000
160,000	20,000	40,000	220,000
400,000	50,000	100,000	550,000
560,000	70,000	140,000	770,000
800,000	100,000	200,000	1,100,000

OneMoreTest trains the neural network with the dataset generated in Section II-B. *OneMoreTest* feeds input (i.e., ipt_i in Equation 2) into the network, expecting it to generate output act_i in Equation 3. If the output of the network is different from act_i, *OneMoreTest* tunes parameters within the network according to the difference between its actual output and the expected output (i.e., act_i) by the well-known backpropagation algorithm [16]. Although the training data may include fault-triggering tests, the neural network is designed to learn general patterns and behaviors from the data, with the ability to ignore noise [17], [18]. If most of the test cases are not fault-triggering, it is likely that a very small percentage of fault-triggering tests would be taken by the neural network as noise data. We train the model with pairs of input-output of the method under test (MUT), and thus the model should learn how to predict the output of the MUT according to given input. Notably, the proposed approach is specially designed to detect hard-to-detect defects where the correct input-output pairs are dominating, i.e., much more popular than incorrect input-output pairs. Consequently, the model should learn the correct functionality much better than the incorrect one. When fault-triggering tests are dominating (i.e., the bug is easy to detect), it would significantly

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

influence the resulting neural network. This is why we suggest that the proposed approach is better suited for identifying hard-to-detect defects.

Notably, training a deep neural network over multiple epochs often results in more accurate network models [19]. Consequently, we keep training the generated network with the generated tests until the loss of the network is smaller than a dynamically determined threshold based on the validation set or until the allocated time slots (empirically set to 30 minutes) are exhausted. The loss threshold is calculated dynamically for each method by monitoring the performance on the validation set during training. Specifically, the threshold is defined as the median loss observed across all batches of the validation set in the first epoch. This ensures that the threshold adapts to the specific characteristics of the method being trained. If the network fails to achieve a loss below this threshold, the model is considered to have failed to fit the data, and no further suggestions are provided for that method. This approach ensures that the neural network is only deployed when it demonstrates sufficient convergence and reliability.

We also empirically determine the maximum number of training epochs. Notably, with the increasing of epochs, the model is more likely to overfit, and thus we should set a maximal number of epochs to avoid overfitting. Our initial evaluation on validating dataset suggests that 3 (epochs) is a balanced setting. Besides the maximal number of epochs, we take the following measures to prevent over-fitting: parameter regularization [20] and dropout [21]. For parameters within the same network, the larger a parameter is, the greater impact it has on the performance of the network. Parameter regularization [20] is to add a penalty term to large parameters to prevent them from dominating the network. Dropout [21] is a strategy to discard neurons in hidden layers randomly during training, which prevents neurons from co-adapting too much. Both parameter regularization and dropout have been proved effective in preventing over-fitting of deep neural networks [20], [21].

The default loss function of the network computes the mean square error (MSE) between the actual output (vectors) of the network and the expected output (vectors). Although MSE has been widely used to measure the performance of neural networks, it does not exactly measure the distance between the actual output and the expected output. To this end, we define the following loss function:

$$loss = \frac{1}{n}\sum_{i=1}^{n}(Distance(act_i, \hat{act_i}))^2 \tag{4}$$

$$Distance(act_i, \hat{act_i}) = \frac{1}{m}\sum_{j=1}^{m} D(opt_{i,j}, \hat{opt_{i,j}}) \tag{5}$$

$$D(opt_{i,j}, \hat{opt_{i,j}}) = \begin{cases} \frac{ED(opt_{i,j}, \hat{opt_{i,j}})}{|opt_{i,j}|+|\hat{opt_{i,j}}|}, \text{if } opt_{i,j} \text{ is textual} \\ \frac{opt_{i,j}-\hat{opt_{i,j}}}{\max\limits_{k\in[1,n]} opt_{k,j} - \min\limits_{k\in[1,n]} opt_{k,j}}, \text{otherwise} \end{cases} \tag{6}$$

where n is the size of the training data, and m is the size of the output. act_i (as defined in Equation 3) and $\hat{act_i}$ are the actual output and estimated output (i.e., output of the network), respectively. $\max_{k\in[1,n]} opt_{k,j}$ and $\min_{k\in[1,n]} opt_{k,j}$ are the maximal and minimal values of the given output parameter, respectively. $ED(opt_{i,j}, \hat{opt_{i,j}})$ is the edit distance [22] between two strings, and $|str|$ is the length of the string str. Compared to MSE, our loss function has the following advantages. First, our loss function facilitates a unified loss-based assessment of different networks. MSE accounts the absolute distance between the actual output and estimated output, and thus the loss is significantly influenced by the ranges of the output. Considering that the output of different networks differs from each other significantly, our loss function replaces the absolute errors with relative errors. Second, our loss function provides a more accurate and meaningful measurement of differences between various data types by applying specialized methods suited to the type of data.

MSE measures the difference between two strings (actual output and estimated output) by computing the difference between two integer sequences. However, the distance between two ASCIIs is meaningless: Although 'i' and 'h' have close ASCII values, generating 'i' is not more accurate than generating 'j' when 'h' is expected. If the resulting network does not converge, *OneMoreTest* fails to simulate the method under test. In this case, OneMoreTest terminates and fails to generate any suggestions for the method.

E. Ranking of Suspicious Tests

If the resulting neural network is convergent, *OneMoreTest* leverages it to distinguish the most suspicious tests. *OneMoreTest* automatically generates testing data by running the mutated tests again. We may keep running the tests until we have enough test cases where each item follows the format defined in Equations 1-3.

From such numerous test cases, *OneMoreTest* should select fault-revealing ones. A test is fault-revealing if its actual output is different from its expected output. *OneMoreTest* computes the suspicion scores for all of the automatically generated tests. The suspicion score of a test case equals the loss of the network when the test case is fed into the network. Specifically, the proposed approach identifies suspicious test cases based on their prediction error, which is calculated as the loss between the neural network's predicted output and the actual output of the MUT. The loss values are then used to rank the test cases from highest to lowest, with the top-ranked test cases being considered the most suspicious. *OneMoreTest* ranks tests in descending order by suspicion scores and presents only the top k most suspicious tests to developers for manual verification. If any of the manually verified tests reveal defects within the method, developers should fix the defects and repeat unit testing on the fixed version of the method.

III. Evaluation

A. Research Questions

The evaluation investigates the following research questions:

- **RQ1**: Is *OneMoreTest* accurate in suggesting fault-revealing unit tests?

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

- **RQ2**: Does *OneMoreTest* outperform existing approaches in suggesting fault-revealing unit tests?
- **RQ3**: Is *OneMoreTest* efficient?

RQ1 concerns the performance of *OneMoreTest*, i.e., how well it works in identifying fault-revealing tests. We measure its performance by precision and recall:

$$precision@k = TruePositives@k \div k \tag{7}$$

$$recall@k = \frac{IdentifiedFaultyMethods@k}{FaultyMethods} \tag{8}$$

where $precision@k$ is the precision when the top k tests are recommended to be validated, and $TruePositives@k$ is the number of true positives (i.e., fault-revealing tests that are ranked in top k by *OneMoreTest*). $recall@k$ is the recall when the top k tests are recommended to be validated, and $IdentifiedFaultyMethods@k$ is the number of faulty methods that are successfully identified by verifying the top k tests suggested by *OneMoreTest*. $FaultyMethods$ is the total number of faulty methods involved in the evaluation. It is worth noting that the denominators of precision and recall are different, which may lead to significant differences in their values. Specifically, recall is computed at the method level: for each method under evaluation, if at least one of the top-k tests reveals the fault, it is counted as a successful identification. Thus, the denominator for recall is the number of faulty methods. In contrast, precision is computed at the test-case level: for each of the N methods, k test cases are evaluated, resulting in a total of $k \times N$ test cases. Therefore, the denominator for precision is $k \times N$, which is typically much larger than the number of methods. This structural difference explains why recall values may appear much higher than precision values in the same top-k setting.

RQ2 concerns the comparison against existing approaches. We compared *OneMoreTest* against the following baseline approaches:

- Random-Selection: Randomly select k tests for validation.
- Coverage-Based Selection: Select the test that results in the maximum increase of code coverage, and repeat the selection until k tests are selected for manual validation.
- Invariant-Based Selection [23]: Pick up the top k tests (if there are any) that violate the most invariants.
- AGA-Based Selection [24]: First select the test case that covers the most code elements (i.e., lines of code), and then iteratively select the test that covers the most uncovered lines until all lines are covered or k tests are selected.
- EvoSuite [4]: For each top-k setting (e.g., top-1, top-10, top-100), we independently invoke EvoSuite k times to generate k distinct test cases for each method. These test cases are executed on both the buggy and fixed versions of the MUT in Defects4J. A test case is considered fault-revealing only if its outputs differ between the two versions. The top-k generated tests are then used to compute precision and recall.

Random-selection is selected because it frequently serves as a baseline in related work [25], [26], [27]. To mitigate the randomness, we repeated the random selection process 100 times and reported the average precision and recall. Coverage-based selection has been widely used to maximize code coverage with the fewest tests based on the assumption that higher coverage of production code often leads to a greater chance of finding defects. In the evaluation, we maximized the coverage of statements in faulty methods. Invariant-based selection is based on the assumption that tests violating identified invariants often reveal defects of the system under test. In the evaluation, we leveraged Daikon [23] to detect invariants automatically because existing evaluations suggest that it is effective and efficient.

AGA-Based Selection [24] is built on the traditional Greedy Additional (GA) algorithm, which iteratively selects the test case covering the most new code elements (i.e., lines of code) until all lines are covered or k tests are selected. AGA enhances GA by using additional data structures to reduce redundant data accesses and optimize the iteration process, significantly improving efficiency while maintaining high fault detection effectiveness. This approach ensures that the test cases selected provide maximum code coverage with improved computational performance. In our implementation of both coverage-based selection and AGA-based selection, the "code elements" refer to lines of code (line coverage). We acknowledge that line coverage is considered weaker compared to other forms of code coverage (e.g., branch coverage, path coverage, or function coverage) in terms of the likelihood of revealing faults. However, in the context of our unit tests at the function level, which involve relatively small methods, line coverage is appropriate. Other coverage metrics like branch coverage or path coverage are less suitable in this context because they can easily reach high values (e.g., 100% coverage) with minimal test cases, providing limited differentiation among test cases. Line coverage offers a more granular measure for our purposes, allowing for meaningful comparison and selection of test cases.

EvoSuite [4] is a mature, state-of-the-art automated unit test generation tool for Java. It uses search-based techniques, notably genetic algorithms (GAs), to evolve test suites towards high coverage criteria—such as line, branch, and mutation coverage. Specifically, EvoSuite starts with a population of random test suites, and applies GA operations like selection, crossover, mutation, and elitism to iteratively improve coverage. It also incorporates optimizations such as DynaMOSA [28], archive-based storage of valuable tests, test minimization, and automatic assertion generation. EvoSuite has been widely adopted in both research and industry due to its effectiveness in quickly producing high-coverage JUnit tests.

RQ3 concerns the efficiency of *OneMoreTest*, i.e., how long it may take to make suggestions for a single faulty method.

B. Dataset

To create a dataset for the evaluation, we selected real-world bugs from the well-known bug repository Defects4J (version 1.0.0) [9]. We selected Defects4J because of the following reasons. First, it is publicly available, facilitating researchers to repeat the evaluation. Second, all of the defects in the repository

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

are accompanied by bug reports and concise patches, facilitating manual and automated validation of the tests. The dataset was constructed with the following reproducible process:

- First, we retrieved all faulty methods in Defects4J (version 1.0.0), which contains 355 real-world bugs from 5 open source applications.
- Second, we excluded methods that use external files as input or output because the current implementation of *OneMoreTest* could not yet automatically generate such complex input. A typical example is *fromCode (String fileName, String code)* from project *Closure*, where the input includes the actual file name.
- Third, we excluded faulty methods whose error rates were 100%. Notably, the *error rate* equals the number of failed test cases divided by the total number of generated test cases. As introduced in Section I, *OneMoreTest* assumes that the developers have already used implemented and passed at least one test case and thus the error rate should be less than 100%. The method we used to calculate error rate is as follows: we run the same test on the buggy and the fixed version. If it produces different outputs on the two versions, it is a "fault-revealing test case". The proportion of "fault-revealing test cases" out of all test cases is called *error rate*.

As result of the construction, we selected 178 faulty methods from 5 projects for the evaluation. In Table II, we summarized the selection and the number of faulty methods excluded for different reasons. Notably, if we search for faulty methods in other bug repositories, the ratios of excluded methods could be different because the construction of bug repositories often has high uncertainty and subjectivity. To clarify, we did not use all developer-written test cases, nor did we use the entire set of "relevant" test cases identified by Defects4J. First, we obtained the buggy method's source code from the unpatched version. Second, we conducted a global search to identify all test cases exercising the buggy method, but excluded any test cases listed in the trigger-tests folder. These excluded test cases happen to reveal the bug but would not be available in a real-world scenario prior to the discovery of the bug. Third, from the remaining test cases, we selected the first passing test case for each buggy method and applied input mutations to generate a large number of test cases. This approach ensures that our evaluation closely aligns with realistic scenarios, where developers lack prior knowledge of bug-revealing test cases and rely on exploratory testing to uncover potential defects. In Fig. 2, we visualized the size (LOC) of the selected faulty methods, as well as the number of methods invoked by the faulty methods. *Invoked Methods Per Faulty Method* specifies how many methods have been directly invoked by a single faulty method. From this figure, we observe that the size of the faulty methods varies substantially from 2 to 223 LOC, with an average of 39.7. From this figure, we also observe that the faulty methods often call other methods (on average 10.8 methods), which may substantially increase the complexity of the faulty method's functionality. If all such invoked methods are inlined into the faulty methods, the size of the faulty methods could be increased significantly. Details of the selected methods, as well as those that were excluded, are available online [29].

TABLE II
SELECTING FAULTY METHODS FROM DEFECTS4J (V1.0.0)

Items	Values
# Faulty methods in Defects4J (v1.0.0)	355
# Faulty methods excluded because of external files	30
# Faulty methods excluded because of 100% error rate	147
# Faulty methods selected for evaluation	178

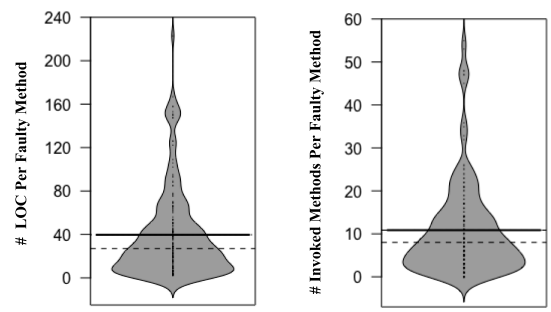

Fig. 2. Complexity of selected methods (bean plot).

C. Process

We evaluated *OneMoreTest* on each of the selected faulty methods as follows.

- First, we leveraged *OneMoreTest* to generate 1,100,000 input-output pairs for each method. These data were deterministically split into three non-overlapping subsets: 800,000 samples for training, 200,000 for validation, and 100,000 for testing.
- Second, the training subset was used to train the model in *OneMoreTest*. If *OneMoreTest* terminated due to *inaccurate simulation*, the evaluation on the corresponding method was terminated.
- Third, the trained network was applied to the 100,000 test cases to select the top-k items based on prediction confidence.
- Fourth, we validated whether the selected items were fault-revealing. An item was considered fault-revealing if and only if the current (buggy) version and the fixed version of the method (from the selected bug repository) produced different outputs for the same input.

The evaluation of the baseline approaches was conducted similarly on the same dataset. Notably, *OneMoreTest* and the baseline approaches shared the same setting, which facilitates comparison among the evaluated approaches. We generated 1,000,000 test cases after the pilot experiment and careful consideration. On the one hand, our pilot experiment showed that using a smaller set (e.g., 100,000 samples) led to both low precision and recall, suggesting that the model struggled to capture sufficient patterns from the data, potentially due to the limited variety in the training set. On the other side, the pilot experiment also suggested that larger training set significantly increased computational costs, including higher memory usage and longer

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

```
public int translate (final CharSequence input, final int index, final
        Writer out) throws IOException {
    ...
    + if (entityValue >0xFFFF) {
        + final char[] chars = Character.toChars(entityValue);
        + out.write(chars[0]);
        + out.write(chars[1]);
    + }else {
        out. write (entityValue );
    +}
    return 2 + end − start + (isHex ? 1 : 0) + (semiNext ? 1 : 0);
}
```

Listing 7. Key logic of the `translate()` method

```
// Passing case (outputs are the same in both buggy and fixed versions):
Input: "Hello 1;;25;59 World"
Buggy Output: "Hello 1;;25;59 World"
Fixed Output: "Hello 1;;25;59 World"

// Failing case (outputs differ between buggy and fixed versions):
Input: "Smiley: 2#16594#"
Buggy Output: "Smiley: 216594"
Fixed Output: "Smiley: 2#16594#"
```

Listing 8. Passing and failing test case examples

training times. The sizes of the training set and testing set are empirically set according to the pilot experiment, balancing the cost and effect. For invariant-based selection, we inferred invariants of a faulty method by Daikon with items in the training data, and sorted items in the testing data according to the number of violated invariants, detailed experimental setup can be found in Section III-E. Random-selection, coverage-based, AGA-based selection and EvoSuite do not request training, and thus we applied them directly to the test cases. All experimental data can be found in the online package, the experimental data for *OneMoreTest* is available at [30], and the replication data for all baselines can be found at [31].

Illustrative Example: To further demonstrate the effectiveness of *OneMoreTest* in identifying fault-revealing test cases, we present an example from our evaluation on Lang-28 in Defects4J. The MUT is `translate(final CharSequence input, final int index, final Writer out)`, which parses character references like "A" and writes the corresponding character (i.e., A) to the output stream.

Listing 7 shows the core logic of the method in the fixed version. The lines prefixed with "+" and highlighted in green indicate the code newly added in the fixed version to correctly handle Unicode codepoints greater than `0xFFFF`. These additions convert the codepoint into a surrogate pair and write it in two steps using `Character.toChars(...)`. The buggy version does not contain these lines and writes the codepoint directly using `out.write(entityValue)`, which may cause incorrect behavior when processing high Unicode values.

Listing 8 presents two representative test cases. The first test case, with input "Hello 1;;25;59 World", produces identical outputs in both buggy and fixed versions, and thus **does not reveal the bug**. In contrast, the second test case "Smiley: 2#16594#" leads to divergent outputs: the buggy version returns "Smiley: 216594", which is an incorrect transformation, while the fixed version correctly preserves the input as "Smiley: 2#16594#". This failing test case clearly **reveals the bug** and was ranked among the top recommendations by *OneMoreTest*, illustrating its ability to effectively prioritize fault-revealing inputs.

D. RQ1: OneMoreTest Is Accurate

To answer RQ1, we applied *OneMoreTest* to the 178 real-world faulty methods selected in Section III-B. The overall results are presented in Tables III and VI whereas detailed evaluation results on each faulty method are presented as a online table [32] for space limitation. "Pr.@k" denotes Precision@k, and "Rc.@k" denotes Recall@k. Precision@k measures the proportion of correctly identified fault-revealing tests among the top-k selected tests. Recall@k measures the proportion of all actual faults that are identified within the top-k selected tests. It is important to note that out of the total 178 faults used in the evaluation, 8 faults were not successfully fitted by the neural network model. Consequently, precision@k was calculated based on the 170 faults that were successfully fitted, excluding these 8 faults. On the other hand, recall@k was calculated considering all 178 faults, treating the 8 unfitted faults as false negatives. This approach ensures that precision reflects the model's performance on the faults it could process, while recall accounts for the entire set of faults, highlighting the model's ability to detect all potential faults. Tables IV and V present the evaluation results under various scales of training data. Specifically, Table IV details how precision and recall (Top@1, Top@10, and Top@100) evolve as the training dataset ranges from 10,000 to 700,000 samples, while Table V highlights the corresponding changes in code statement coverage (up to 1,000,000 samples). Together, these tables illustrate the relationship between dataset size and the model's performance in terms of accuracy and completeness. Notably, for each faulty method, we generated 100,000 test cases, and the top k were selected by the proposed approach from such a large number of candidates. Consequently, top 1 and top 100 are equivalent to top 0.001%=1/100,000 and top 0.1%=100/100,000 respectively.

From the evaluation results, we make the following observations:

- First, *OneMoreTest* is often accurate. In 130 of the 178 faulty methods, the most suspicious items suggested by *OneMoreTest* are fault-revealing, resulting in a recall@1 (i.e., recall when k=1) of 70.79%=126/178. For 120 methods, all of the top 100 suggested items are fault-revealing. Such a high accuracy suggests that in most cases (70.79%) developers can find the faulty MUT by checking only a single test suggested by *OneMoreTest*. Notably, only a small part (23.96% on average) of the items are fault-revealing. Consequently, it is challenging to accurately pick up a fault-revealing item surrounded by a large number of items (which cannot reveal faults). *OneMoreTest*, however, successfully picked up fault-revealing items, which may significantly improve the chance in identifying

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EVALUATION RESULTS

	OneMoreTest			Random Selection			Coverage-based Selection			Invariant-based Selection			AGA-based Selection		
	Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100
Pr.@k	72.62%	74.71%	74.12%	22.03%	22.16%	22.67%	16.29%	33.65%	31.96%	44.51%	46.19%	44.08%	46.63%	58.65%	58.70%
Rc.@k	70.79%	79.78%	83.71%	22.03%	71.11%	97.44%	16.29%	92.13%	99.40%	43.2%	90.45%	97.19%	46.63%	75.28%	75.84%

TABLE IV
EVALUATION RESULTS FOR VARIOUS SCALES OF TRAINING DATA

	Test Case Set Size														
	10,000			100,000			200,000			500,000			700,000		
	Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100
Pr.@k	62.41%	61.12%	55.81%	61.24%	62.81%	60.64%	61.93%	63.24%	61.41%	63.47%	64.73%	64.13%	63.69%	64.88%	64.92%
Rc.@k	62.41%	74.16%	85.39%	61.24%	71.91%	80.34%	61.24%	71.91%	81.46%	59.55%	69.66%	75.28%	60.11%	70.22%	78.65%

TABLE V
CODE STATEMENT COVERAGE FOR VARIOUS SCALES OF TRAINING DATA

Project	**#All / Used Bugs**	**Original Coverage**	**Avg.Coverage (10,000)**	**Avg.Coverage (100,000)**	**Avg.Coverage (200,000)**	**Avg.Coverage (500,000)**	**Avg.Coverage (700,000)**	**Avg.Coverage (1,000,000)**
Chart	26 / 15	86.46%	92.34%	95.57%	97.53%	97.53%	97.53%	97.53%
Closure	133 / 48	93.65%	95.21%	99.40%	99.40%	99.40%	99.40%	99.40%
Lang	63 / 45	85.76%	89.12%	95.34%	99.12%	99.12%	99.12%	99.12%
Math	106 / 48	93.58%	98.77%	98.92%	98.92%	98.92%	98.92%	98.92%
Time	27 / 22	83.32%	88.82%	93.82%	98.82%	98.82%	98.82%	98.82%
All	355 / 178	90.16%	95.32%	98.47%	98.53%	99.01%	99.01%	99.01%

hard-to-detect defects in unit testing. Notably, the upper limit of *OneMoreTest*'s recall is significantly influenced by the 8 out of 178 faults that the neural network model failed to fit successfully. Consequently, "Precision@1" was calculated based on the 170 faults that were successfully fitted, excluding these 8 faults. In contrast, "Recall@1" was calculated considering all 178 faults, treating the 8 unfitted faults as false negatives. This discrepancy in the denominators used for precision and recall calculations leads to the observed differences in "Precision@1" and "Recall@1".

- Second, When increasing the dataset size from approximately 100,000 to 1,000,000 samples, the model achieves a more favorable balance between accuracy and coverage. Specifically, as the data volume expands, the Top@1 precision improves from 61.24% to 72.62%, and the recall increases from 61.24% to 70.79%. These trends indicate that at the million-sample scale, the model not only predicts targets more accurately but also maintains a high degree of comprehensiveness, thereby striking a more optimal balance between precision and coverage. Therefore, selecting a training dataset of around one million samples is clearly a more advantageous strategy. Moreover, coverage increases notably as the training dataset grows. Specifically, our experiments show that the average statement coverage across all projects rises from 98.32% at 100,000 samples to 99.01% at 1,000,000 samples. Although this gain appears incremental in percentage terms, it indicates a broader range of tested code segments. However, coverage plateaus around 500,000 samples, suggesting limited additional benefit beyond that point. Nonetheless, at the million-sample level, the model achieves both higher accuracy (as reflected by precision and recall) and near-comprehensive coverage, underscoring the advantage of selecting approximately one million training samples for an optimal balance between precision and coverage.
- Third, the proposed approach keeps accurate regardless of the error rates. From Table VI, we observe that the minimal precision@1 is 60.29% and the minimal recall@1 is 60.29%, suggesting that regardless of the changes in error rate the proposed approach can generate fault revealing test cases accurately. We also observe that when the error rate increased from (0%, 30%] to (50%, 100%], the precision@1 decreased from 74.56% to 73.27%. However, we also notice that when the error rate is smaller than 50%, its decrease (i.e., increasing from (0%, 30%] to (30%, 50%]) does not substantially influence the performance of the proposed approach, suggesting that the proposed approach works well. This is primarily because a significant portion of the bugs in the methods with higher error rates are exception-type bugs. In our approach, we treat these exception-type bugs as special cases. Specifically, in the test case generation phase, we mark test cases that throw unexpected exceptions with the special string "ERROR$" to distinguish them from those that are expected to throw exceptions. During the testing phase, these anomalous cases are prioritized by assigning them an exceptionally large distance value (e.g., 1e9), ensuring that they are ranked at the top of the recommendation list. While this strategy helps in prioritizing exceptional cases,

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
PERFORMANCE ON FAULTY METHODS WITH DIFFERENT ERROR RATES

Error Rate	Metric	OneMoreTest			Random Selection			Coverage-based Selection			Invariant-based Selection			AGA-based Selection		
		Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100
(0,0.3]	Pr.@k	74.56%	76.55%	76.49%	10.98%	11.33%	11.65%	6.35%	26.51%	22.87%	36.59%	41.14%	38.01%	51.59%	64.76%	64.8%
	Rc.@k	75%	79.64%	86.93%	10.98%	73.11%	97.45%	6.35%	88.89%	99.21%	35.71%	88.89%	97.62%	51.59%	77.78%	77.78%
(0.3,0.5]	Pr.@k	60.29%	66.54%	61.34%	37.21%	36.98%	37.35%	35.71%	46.07%	48.68%	51.85%	47.41%	51.63%	35.71%	47.86%	46.29%
	Rc.@k	60.29%	75.67%	78.67%	37.21%	99.68%	100%	35.71%	100%	100%	50%	96.43%	96.43%	35.71%	78.57%	82.14%
(0.5,1)	Pr.@k	73.27%	74.16%	74.57%	62.33%	61.75%	63.38%	45.83%	56.67%	60.17%	78.26%	71.74%	70.26%	33.33%	39.17%	41.13%
	Rc.@k	75%	75%	75%	62.33%	87.50%	99.88%	45.83%	100%	100%	75%	91.67%	95.83%	33.33%	58.33%	58.33%

TABLE VII
PERFORMANCE ON FAULTY METHODS WITH DIFFERENT ERROR RATES WITHOUT EXCEPTION

Error Rate	Metric	OneMoreTest			Random Selection			Coverage-based Selection			Invariant-based Selection			AGA-based Selection		
		Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100	Top1	Top10	Top100
(0,0.3]	Pr.@k	65.19%	68.16%	68.67%	11.63%	12.07%	12.48%	4.76%	24.76%	22.31%	34.15%	39.39%	35.18%	47.62%	66.91%	66.36%
	Rc.@k	60.28%	75.87%	83.12%	11.63%	62%	97.51%	4.76%	88.1%	100%	33.33%	86.90%	97.62%	47.62%	80.95%	80.95%
(0.3,0.5]	Pr.@k	40.56%	49.33%	45.04%	39.5%	37.87%	38.23%	50%	49.44%	51.83%	41.18%	43.53%	45.18%	22.22%	43.33%	42.67%
	Rc.@k	40.56%	62.67%	68.55%	39.5%	98.83%	100%	50%	100%	100%	38.89%	94.44%	94.44%	22.22%	77.78%	83.33%
(0.5,1)	Pr.@k	49.78%	47.89%	47.92%	60.4%	60.93%	61.11%	40.0%	50.67%	54.46%	78.57%	69.29%	67.0%	40.0%	50.67%	52%
	Rc.@k	46.67%	46.67%	46.67%	60.4%	85.73%	99.80%	40.0%	100%	100%	73.33%	86.67%	93.33%	40.0%	73.33%	73.33%

the high proportion of such bugs in the test set makes the overall performance of the approach less effective under high error rates. The experimental results after excluding these exception-related bugs are shown in Table VII. After removing exception-type bugs, the precision and recall in the higher error rate range (0.5, 1] are not as good, and even worse than some baselines. This suggests that our method is more effective in the lower error rate ranges. Since we handle exception-type bugs in a special way, we can consistently rank these test cases well. However, for non-exception-type bugs, which are logic faults, we recommend introducing manual feedback. We will provide a more detailed explanation in Section IV-B.

- Fourth, *OneMoreTest* may fail occasionally. In 23 out of the 178 faulty methods, none of the top 100 suspicious tests suggested by *OneMoreTest* are fault-revealing, suggesting that *OneMoreTest* did not work on these methods. If only top 1 test case were considered, *OneMoreTest* failed to identify fault-revealing test cases on 29.21%=1-70.79% of the evaluated methods. All such figures suggest that it remains challenging to generate and select fault-revealing tests to trigger hard-to-find bugs.
- Fifth, *OneMoreTest* failed to simulate the tested functionality on 8 out of 178 faulty methods, accounting for 4.49%=8/178 of the evaluated methods. In such cases, *OneMoreTest* did not make any suggestions. Notably, *OneMoreTest* failed in these cases because the neural network was unable to accurately simulate the target methods (that is, its loss, evaluated on the validation set, exceeded the dynamically determined threshold based on the training and validation process).

We take the faulty method M41 (*boolean isSameLocalTime(Calendar cal1, Calendar cal2)*) [33] (as presented in Listing 9) to illustrate why *OneMoreTest* works. Notably, the faulty lines are marked with red background and notated with a special initial letter "-". In contrast, the fixed version of the buggy lines are marked with green background and the nitial letter "+". The faulty method *isSameLocalTime* is used to compare whether two *Calendar* objects equal. It achieves this by comparing the data type of the objects, and validating the equivalence concerning their key fields, e.g., *MILLISECOND*, *SECOND*, *MINUTE*, *HOUR*, *DAY_OF_YEAR*, *YEAR*, *ERA*. Notably, the faulty version validates the equivalence concerning the field *HOUR* (Line 7). However, *HOUR* employs a 12-hour system, and thus the comparison could be inaccurate. For example, 4 o 'clock would equal 16 o 'clock if the comparison is conducted within a 12-hour system. To fix the bug, we should replace the field *HOUR* with another field *HOUR_OF_DAY* that is presented in a 24-hour system (Line 8). From the automatically generated training dataset, *OneMoreTest* found that in most (95.87%) of the cases the two to-be-compared objects (i.e., *cal1* and *cal2*) have equivalent *HOUR_OF_DAY* when the output of the to-be-tested method returns *true*. Consequently, when the fault-revealing test case (where the two to-be-compared objects have nonequivalent *HOUR_OF_DAY* whereas the output of the to-be-tested method returns *true*), *OneMoreTest* successfully reported the test case as suspicious. This example may suggest that *OneMoreTest* is good at mining useful mappings between input and output, and is also good at identifying suspicious test cases that violate the mappings. This is one of the reasons why *OneMoreTest* is effective.

We take M161 (*Complex divide(Complex divisor)*) [34] (as presented in Listing 10) to illustrate why *OneMoreTest* fails occasionally. This method is used to perform the division of complex numbers. It first checks whether *dividend* or *divisor* is *NaN* (i.e., not a number). If yes, it returns *NaN* (Lines 3-5).

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

```
public static boolean isSameLocalTime(Calendar cal1, Calendar cal2) {
    ...
    return (
    cal1.get(Calendar.MILLISECOND) == cal2.get(Calendar.
            MILLISECOND) &&
    cal1.get(Calendar.SECOND) == cal2.get(Calendar.SECOND) &&
    cal1.get(Calendar.MINUTE) == cal2.get(Calendar.MINUTE) &&
    - cal1.get(Calendar.HOUR) == cal2.get(Calendar.HOUR) &&
    + cal1.get(Calendar.HOUR_OF_DAY) ==
    cal2.get(Calendar.HOUR_OF_DAY) &&
    cal1.get(Calendar.DAY_OF_YEAR) == cal2.get(Calendar.
            DAY_OF_YEAR) &&
    cal1.get(Calendar.YEAR) == cal2.get(Calendar.YEAR) &&
    cal1.get(Calendar.ERA) == cal2.get(Calendar.ERA) &&
    cal1.getClass() == cal2.getClass());
}
```

Listing 9. Typical Faulty Method (M41)

```
1  public Complex divide(Complex divisor)
2      ...
3      if (isNaN || divisor.isNaN) {
4          return NaN;
5      }
6
7      - if (divisor.getReal() == 0.0 && divisor.getImaginary() == 0.0) {
8          - return NaN;
9      + if (divisor.isZero) {
10         + return isZero ? NaN : INF;
11     }
12
13     if (divisor.isInfinite() && !isInfinite()) {
14         return ZERO;
15     }
16     ...
17 }
```

Listing 10. Typical Faulty Method (M161)

Then it validates whether both the real and imaginary of the *divisor* equal 0. If yes, the method returns *NaN* (Lines 7-8). The rationale is that zero cannot be used as a divisor. If the *divisor* is infinity and the *dividend* is not infinity, the method returns zero (Lines 13-15). In the faulty version, if the *divisor* equals zero, the method would return *NaN*. However, in the fixed version, if both *divisor* and *dividend* equal zero, the method should return INF (infinite) instead of NaN (Lines 9-10). Notably, none of the test cases run on the faulty version returned INF, a particularly special value. This is because the code in the faulty version never returns INF, and as a result, the neural network trained on these test cases failed to learn that INF should be returned in certain scenarios. In the fixed version, however, INF is the expected output when both the divisor and dividend are zero. Consequently, the network failed to identify the fault-revealing test cases where INF (rather than NaN) should have been returned.

We notice that in 8 out of 178 faulty methods (noted as M171-178), *OneMoreTest* failed to obtain a convergent deep neural network. Following the criterion dynamically determined during training, we deemed a network as not converging if its average loss remained greater than the dynamically determined threshold on the validation set, regardless of the increasing number of training iterations. The average loss on the 8 methods (i.e., M171-178) were 0.2754, 0.2216, 0.2213, 0.2087, 0.2713, 0.2104, 0.2436 and 0.2314, respectively, all exceeding their corresponding validation loss thresholds. In contrast, the loss on other methods varied from 3.6e-9 to 0.0919, substantially smaller than the dynamic thresholds. A possible reason for the large loss on M171-M178 is that such methods are substantially more complex than the majority of Java methods.

A typical example is M174 (*VectorialPointValuePair doOptimize()*) [35] that performs optimization for vector point pairs. The major task of the method is to resolve the nonlinear least squares problem based on Levenberg-Marquardt algorithm [36]. Although the method itself is not lengthy (consisting of 223 lines of source code and 18 branches), it invokes 19 complex methods (containing more than one thousand lines of code in total) defined in the enclosing class. In other words, the functionality of the method in fact is implemented by more than *one thousand* lines of source code. Such a complex algorithm is difficult to simulate accurately with general-purpose fully-connected neural networks without in-depth customization. Notably, such hard-to-simulate methods are not common in Java. In our evaluation, they account for less than five percent (4.49%=8/178) of the evaluated methods, which suggests that they may not significantly reduce the usability of the proposed approach. Based on the preceding analysis, we conclude that *OneMoreTest* is accurate in suggesting fault-revealing items.

E. RQ2: Outperforming Baseline Approaches

To answer RQ2, we compared *OneMoreTest* against existing approaches, including random selection, coverage-based selection, invariant-based selection, AGA-based selection, and EvoSuite.

Random selection is conducted by randomly shuffling the order of candidate test cases and selecting the top-k for evaluation. To mitigate the effects of randomness, we repeated the random shuffling process 100 times and reported the average precision and recall over these 100 runs.

EvoSuite [4] is used as a representative state-of-the-art test generation tool. Although EvoSuite does not perform test prioritization, we follow a controlled evaluation protocol to support fair comparison: for each top-k setting (e.g., top-1, top-10, top-100), we independently invoke EvoSuite k times to generate k distinct test cases for each method. These test cases are then executed on the buggy and fixed versions from the Defects4J benchmark. We determine whether a test case reveals a real bug by observing behavioral differences between the two versions and compute the corresponding precision and recall.

Invariant-based selection is implemented using Daikon [23], a tool that infers likely invariants from program execution traces. The process is conducted as follows:

- First, we use Chicory [37], Daikon's front-end for Java, to instrument the target methods and generate execution traces. Each test case is executed individually to produce its own trace file.
- Next, Daikon analyzes these trace files and infers invariants for each method.
- Then, for each test case, we count how many inferred invariants are violated.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

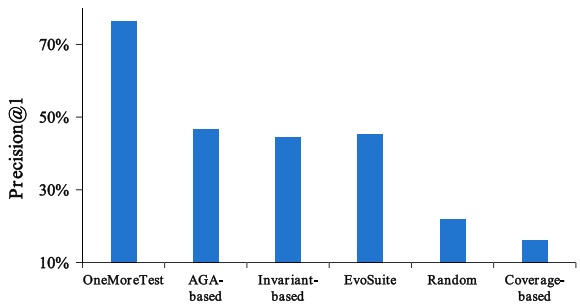

Fig. 3. Improving precision of top@1.

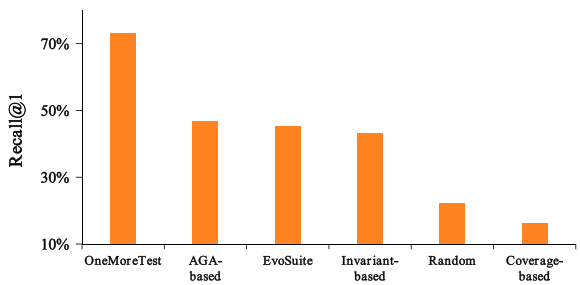

Fig. 4. Improving recall of top@1.

- Finally, the test cases are ranked in descending order based on the number of violated invariants.

AGA-based selection is conducted following the protocol and default settings recommended by the original authors [23]. This method leverages adaptive greedy algorithms to rank test cases based on their potential to cover diverse behavioral characteristics.

Coverage-based selection ranks test cases by their individual code coverage scores, computed using statement-level coverage. The top-k tests with the highest coverage are selected.

The comparison results are presented in Tables III and VI, while Figs. 3 and 4 visualize the comparative performance across all baseline methods. From the tables and the figures, we make the following observations:

- **First,** *OneMoreTest* substantially outperforms the baseline approaches concerning both precision and recall. For example, its precision@1 is 72.62%, substantially higher than those of the random selection (22.03%), coverage-based selection (16.29%), invariant-based selection (44.51%), AGA-based selection (46.63%) and EvoSuite (45.21%). Its recall@1 (70.79%) is also substantially higher than those of the baseline approaches (varying from 16.29% to 46.63%). Figs. 3 and 4 visualize to what extent *OneMoreTest* improves the precision@1 and recall@1, respectively.
- **Second,** as k increases, the $recall@k$ of the baseline approaches increases significantly while the $precision@k$ remains stable. When k reaches 100, the recall of the baseline approaches surpasses that of *OneMoreTest*. However, the baseline approaches do not outperform *OneMoreTest* in terms of precision. These results suggest that *OneMoreTest* excels at selecting a small number of potentially fault-revealing tests (hence the name *One-More-Test*) and may not be suitable for prioritizing a large number of unit tests.
- **Third,** coverage-based selection and random selection resulted in the poorest performance. It is reasonable that random selection is less accurate because only a small part of the tests are fault-revealing (especially for hard-to-detect bugs). However, it is surprising the coverage-based selection is not much better than random selection. One possible reason is that the involved faulty methods contain few branch nodes, and thus running a small number of tests could maximize the coverage. Once the coverage is maximized, coverage alone could not guide the selection of tests anymore.
- **Fourth,** invariant-based selection substantially outperforms random selection and coverage-based selection. It may suggest that discovering invariants by mining executions of tests is beneficial in picking fault-revealing tests. It also confirms the conclusion in Section V-C that it is promising to derive test oracles by mining the dynamic executions of software.
- **Fifth,** while OneMoreTest outperforms the baselines when k is low (e.g., 1), its recall may be surpassed by alternative approaches when k increases. Both invariant-based and coverage-based selection outperform OneMoreTest for k=10 and k=100 in terms of Recall@k. One possible reason is that the baseline approaches (especially the coverage-based approach) emphasize more on diversity of the selected test cases than our approach. As a result, when k increases, their recall@k increases significantly.
- **Sixth,** Table VIII summarizes the comparison results between *OneMoreTest* and EvoSuite in terms of precision, recall, and average coverage across different top-k. From Table VIII, we make the following observations: **First**, *OneMoreTest* consistently achieves higher precision than EvoSuite for all three top-k settings. For instance, at Top1, the precision of *OneMoreTest* is 72.62%, compared to EvoSuite's 45.21%. This indicates that when selecting only a single test (i.e., $k = 1$), *OneMoreTest* is more likely to pick a fault-revealing test than EvoSuite. **Second**, regarding recall, *OneMoreTest* performs better at Top1 (70.79% vs. 45.21%), but is surpassed by EvoSuite at Top10 and Top100 (79.78% vs. 87.08%, and 83.71% vs. 93.26%). This aligns with the earlier observation that *OneMoreTest* particularly excels at finding a small number of fault-revealing tests, whereas EvoSuite's benefits from generating a larger suite of tests to maximize recall. **Third**, in terms of average coverage, when $k = 1$ or $k = 10$, EvoSuite (89.72% and 95.77%) slightly outperforms *OneMoreTest* (89.19% and 92.83%). However, at $k = 100$, *OneMoreTest* achieves a marginally higher coverage (98.99%) than EvoSuite (97.63%). These results indicate that while EvoSuite can quickly achieve a broader coverage with fewer tests, *OneMoreTest* can maintain competitive or even higher coverage when the number

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
EVALUATION RESULTS FOR ONEMORETEST AND EVOSUITE

Metrics	OneMoreTest			EvoSuite		
	Top1	Top10	Top100	Top1	Top10	Top100
Precision	72.62%	74.71%	74.12%	45.21%	53.73%	58.78%
Recall	70.79%	79.78%	83.71%	45.21%	87.08%	93.26%
Avg. Coverage	89.19%	92.83%	98.99%	89.72%	95.77%	97.63%

of selected tests increases. Overall, these experiments with EvoSuite reinforce our conclusion: *OneMoreTest* is particularly suitable for scenarios where only a few tests can be selected and high-precision fault detection is a priority. Meanwhile, approaches like EvoSuite, which emphasize generating a large number of diverse tests, can yield higher recall when the selection size (k) grows large, albeit often at the cost of lower precision.

- Finally, the error rate has a significant impact on the performance of the evaluated approaches. For example, the precision@1 of random selection increases from 10.98% to 37.21% and 62.33% when the error rate increases from (0%, 30%] to (30%, 50%] (50%, 100%), respectively. Similarly, Coverage-based Selection shows an increase from 6.35% to 35.71% and 45.83% under the same error rate intervals, while Invariant-based Selection sees a smaller improvement, with precision@1 increasing from 36.59% to 51.85% and 78.26%. In contrast, the increase in error rate has a smaller influence on the more advanced approaches, such as OneMoreTest and AGA-based selection. These methods exhibit more stable performance, with smaller fluctuations in precision@1 despite the change in error rate, maintaining their effectiveness across different levels of fault complexity.

We conclude that *OneMoreTest* outperforms baseline approaches in selecting a small number of fault-revealing tests, especially for hard-to-detect defects.

F. RQ3: OneMoreTest Is Efficient

It is time and resource consuming to generate a large number of input-output pairs and to train deep neural networks with such data. To reveal how efficient *OneMoreTest* is, we recorded the training time and testing time for each of the faulty methods. The evaluation was conducted on a personal computer with Intel (R) Core (TM) i7-8700 CPU @ 3.20GHz and 32GB RAM, Windows 10 operation system, Java 8, and Python 3.9.

Our evaluation results suggest that it took *OneMoreTest* only one minute on average to generate a million input-output pairs for a single MUT, and 5 minutes to generate and train a deep neural network with a million input-output pairs. Our evaluation results also suggest that it took *OneMoreTest* less than 10 seconds to select fault-revealing tests from 100,000 candidates for a given method. Notably, in case the network was not convergent, *OneMoreTest* terminated when the maximum time slot (30 minutes) was exhausted.

It should be noted that the performance of *OneMoreTest* could be further improved in future. On one side, we may run *OneMoreTest* on powerful servers instead of a personal computer (as what we did in the evaluation). Equipped with powerful GPUs, the servers can substantially speed up the training and testing of the neural networks, the most time-consuming part of the proposed approach. On the other side, the automated generation of test cases (one million for each faulty method) is inherently parallelable, and thus parallelizing the algorithm in future would substantially speed up *OneMoreTest*.

IV. DISCUSSION

A. Threats to Validity

A threat to external validity is that the proposed approach was evaluated with only 178 faulty methods. Some special characters of such methods may have biased the conclusions drawn on the evaluation. We used the older version of Defects4J for evaluation due to limited budget. The evaluation on Defects4J took us three months. Noting that Defects4J 2.0 is essentially the same as 1.0 except for the increased size, we conducted the evaluation on Defects4J 1.0 only. To reduce the threat, we tried our best to increase the diversity of the dataset. We also created the dataset in a reproducible way. As such, the evaluation could be replicated and even strengthened in future by other researchers.

A threat to construct validity is that the ground truth (i.e., which tests are fault-revealing) could be inaccurate. A test is taken as "fault-revealing" if it leads to different output on the faulty version and the fixed version of the MUT. However, the difference could be caused by the randomness of the method (e.g., random numbers generated and exploited by the method), instead of the bug-fixing. Such tests are well-known as flaky tests [38]. Consequently, such "fault-revealing" tests may fail to reveal faults in practice. To reduce the threat, we manually checked the involved methods to make sure that they are deterministic and their output are not dependent on randomness. In this study, the error rate is calculated as the ratio of incorrect test cases N to the total number of generated test cases M, which "incorrect test cases" refer to those whose actual outputs differ from the expected outputs, as determined by predefined ground truth values. This metric is solely used for experimental purposes to compare the effectiveness and recall of *OneMoreTest* with other baseline methods. In practical applications, the error rate does not provide actionable guidance for developers. Additionally, our approach does not rely on the outputs of fixed versions of methods, as this would contradict typical real-world testing scenarios. Instead, it prioritizes suspicious test cases based on their anomaly scores, allowing developers to focus on the most relevant test cases and efficiently identify potential defects.

B. Limitations

OneMoreTest, as currently implemented, has certain limitations. Specifically, it is unable to handle scenarios where external files are used as input or output. This limitation also applies to data retrieved from databases or the web, as the method is primarily designed to work with test cases generated from the project's codebase. To quantify this limitation, we

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

evaluated *OneMoreTest* across 355 faults. Out of these, 26 faults involved external data and thus could not be processed, which represents approximately 7.3% of the total. This indicates that while *OneMoreTest* can be effective in many cases, there is still a portion of faults where it cannot be applied due to its current constraints. In future work, we plan to extend the capabilities of *OneMoreTest* to support a wider range of input data types, including those sourced from external files, databases, and web services. This enhancement will improve the practical applicability and usefulness of our approach in more diverse real-world scenarios.

Furthermore, if the MUT gives incorrect outputs for more than 70% of the inputs, the neural network may learn incorrect behavior, and thus it becomes less accurate. Notably, in such high-error-rate scenarios, even random selection can achieve high effectiveness, suggesting that the faults are easy to detect. OneMoreTest focuses on scenarios with low error rates (under 30%), where faults are hard-to-detect. Our results demonstrate that in these cases, *OneMoreTest* significantly improves effectiveness by 22.97% (from 51.59% to 74.56%) compared to the best existing methods. In contrast, when the error rate is between 50% and 100%, *OneMoreTest* shows a slight decrease in effectiveness compared to Invariant-based selection, with *OneMoreTest*'s performance at 73.27% (compared to Invariant-based selection's 78.26%). These findings indicate that *OneMoreTest* achieves greater performance improvements in low-error-rate scenarios. It also justifies the position of the proposed approach: It is a useful supplement to existing testing techniques, instead of a replacement of such techniques. In our approach, we treat exception-type bugs as special cases, which is explained in Section III-D. Specifically, these anomalous cases are prioritized by assigning them an exceptionally large distance value (e.g., 1e9), ensuring that they are ranked at the top of the recommendation list. Despite the effectiveness of our approach in detecting exception-type bugs, the model encounters limitations when handling non-exception logic errors, especially in high error rate scenarios. In these cases, the precision and recall in the higher error rate range (0.5, 1] are not effective, and sometimes, they even fall short of baseline methods. To address this issue, we suggest a manual feedback mechanism to improve the effectiveness under these circumstances. We propose periodically reviewing the top 10 recommended test cases to identify any unflagged errors, allowing for model refinement based on observed discrepancies. If inaccuracies persist, we suggest adjusting the model by recommending test cases starting from those with the smallest loss, that is, those with the smallest distance between predicted and actual values. This feedback loop would help capture even subtle errors, preventing them from being overlooked due to insufficient exception flagging.

Another limitation is that the proposed approach requests a large number of test cases that could be expensive to generate and execute. The input-output pairs of such test cases are employed to train the neural network, aiming to learn the functionality of the method under test from the input-output pairs. However, when the method under test is complex, executing the test cases (to obtain the output) could be resource-consuming, which makes the proposed approach less effective in this case. Notably, in our evaluation on 178 real-world methods with a personal computer, the average execution time is 58.1 seconds, the median time is 31.2 seconds and the maximal time is 443.5 seconds, which suggests that the proposed approach is efficient in most cases. Furthermore, since the execution of different test cases is independent and inherently concurrent, the execution time could be significantly reduced by running the approach on a computer cluster or cloud computing servers, which are capable of executing multiple test cases simultaneously.

In real-world projects, fault locations and fault-revealing test cases are typically unknown. Unlike in Defects4J, where such information is predefined and can be directly used for validation, real-world scenarios require developers to construct their own assertions based on the behavior of the MUT. Our approach focuses on generating and prioritizing test cases that are most likely to expose potential faults, rather than providing explicit assertions. Instead, it provides detailed input-output pairs for each recommended test case, prioritizing those most likely to reveal defects. Developers can use this information to analyze the MUT's behavior and construct meaningful assertions based on domain knowledge. This design emphasizes assisting developers by reducing the manual effort required to identify potential defects, while still relying on their expertise to validate the correctness of the outputs. The prioritization of suspicious test cases ensures that the most relevant test cases are presented first, making the process efficient and targeted.

C. More Advanced Networks

To simplify the proposed approach, in this paper, we only leverage the basic fully connected networks. It is potentially fruitful to exploit more advanced neural networks like LSTM and CNN because such advanced networks could be more powerful in simulate the behavior of the MUT [39]. However, more advanced networks are also more likely to overfit, especially when MUTs are small and simple.

Notably, automatic machine learning frameworks, like Google AutoML [40] and Auto-Keras [41] can design advanced and powerful networks automatically according to given training/testing data. Additionally, the latest large language models (LLMs), such as OpenAI's GPT [13] and Codex [42], Google's BERT [43] and T5 [44], and Meta's LLaMA [45], have demonstrated powerful capabilities in natural language processing and generation, providing additional advantages in simulating complex methods. For example, Wang [46] discussed various LLM applications in software testing, including unit test generation, system test input generation, and code error fixing. Although this may require powerful and expensive computing resources and carries the risk of overfitting, exploring such automatic machine learning networks and LLMs in the future is interesting and potentially fruitful. Further research is needed to thoroughly evaluate their effectiveness and mitigate potential overfitting issues.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

D. Bug-Free Methods

The proposed approach, as well as other test case selection and prioritization techniques, does not assume that developers know whether the method under test (MUT) contains bugs. They select the most suspicious test cases regardless of the MUT's correctness. If the MUT is bug-free, the test cases selected by any of these techniques may not reveal any bugs. However, instead of being denied, these test cases could be marked as "not bug-revealing" and kept by developers for future regression testing.

A possible way to prevent *OneMoreTest* from recommending suspicious test cases for bug-free methods is to employ a threshold: Only if the deviation of the suspicious test case is greater than the threshold, the test case should be recommended for manual validation. The deviation of a test case could be measured by the distance between the actual output and expected output (noted as offset) divided by the average offset on the given method.

E. Applicability to Integration and System Testing

Our proposed method focuses on unit testing at the method level, where methods are relatively simple and self-contained. We acknowledge that developers are often more interested in test generation for integration and system levels, where testing is more complex and challenging. Extending our method to higher levels of testing is a promising direction for future work. However, there are practical limitations in its current form. Integration and system-level testing often involve complex interactions, dependencies, and shared states that may not be easily captured by our current approach. Our method relies on collecting large amounts of input-output pairs to train neural networks that simulate the behavior of the method under test (MUT). At higher testing levels, inputs and outputs become significantly more complex, involving multiple interacting components, external systems, or asynchronous behaviors. This complexity poses challenges in generating sufficient and meaningful training data and in designing neural networks capable of modeling such intricate behaviors accurately.

Recent advancements in machine learning, particularly with large models such as deep neural networks and large language models, offer new opportunities to scale our approach to integration and system levels. As discussed in Section IV-C, recent advancements in machine learning, particularly with large models such as transformers and other deep architectures, provide a promising direction for scaling our approach to higher levels of testing. These models have demonstrated the capability to handle significantly more complex patterns, interactions, and data dependencies, potentially enabling more accurate simulations and test case prioritization at higher testing levels. In addition, more sophisticated coverage criteria can be used to improve the effectiveness of the test at these levels. Although our current approach uses line coverage due to the simplicity of the methods under test, integration and system testing benefit from higher-level metrics such as branch coverage, path coverage, data flow coverage, and model-based coverage [47], [48]. These criteria are designed to capture the complex interactions and execution paths characteristic of integrated components and full systems, potentially revealing faults that line coverage might miss [49]. Adapting our method to incorporate these advanced metrics could further enhance its effectiveness in detecting faults in complex software systems.

V. Related Work

A. Test Case Selection and Prioritization

Test case selection (TCS) is to select and execute only a subset of the regression tests aiming to reduce the cost of regression testing [50]. TCS selects such tests on version V_{n-1} whose execution may be influenced by the modification (i.e., the difference between the current version V_n and the original version V_{n-1}). Other tests are not influenced by the modification, and thus it is unlikely for them to reveal new defects caused by the modification. Consequently, if test case selection techniques are properly applied, TCS may significantly reduce the number of executed tests whereas the fault-revealing capability of the regression testing is not significantly reduced [51]. However, it is often challenging and time-consuming to make perfect test case selection, and the cost could be even larger than re-running all test cases [52]. Consequently, various approaches have been proposed to speed up the selection at the cost of reduced safety [52], [53].

Test case prioritization (TCP) is to schedule the execution order of tests to expose errors earlier [54]. TCP executes the most beneficial tests first, expecting such tests to reveal the most defects. For example, testers may execute such tests first that could lead to the maximal increase in the coverage of the regression testing [55], [56]. The key rationale is that higher coverage of the regression testing often results in greater capability in fault detection. Efficiency of TCP algorithms is also a key concern, and thus a few approaches have been proposed to reduce the cost of test case prioritization [24]. Note that various of testing information has been exploited by various TCP approaches, and Ramírez et al. made a taxonomy of such information [57]. According to the taxonomy, the output of systems under test (SUT) is rarely used. Lima et al. [58], proposed COLEMAN, a Multi-Armed Bandit-based approach, addressing TCP challenges in continuous integration. Yaraghi et al. [59] introduced a novel data model and evaluating ML-based TCP techniques with 25 open-source systems. Bagherzadeh et al. [60] pioneered reinforcement learning-based test case prioritization for continuous integration, showcasing improved accuracy and highlighting the potential of reinforcement learning in TCP.

Detailed introduction to test case prioritization is referred to the systematic review made by Khatibsyarbini et al. [54], and detailed introduction to test case selection is referred to the systematic review by Engström et al. [61] and the empirical study conducted by Graves et al. [62].

B. Deriving Test Oracles

A test oracle is a source of information that determines whether the output of a software system (or program or function or method) is correct [7]. Dinella et al. [63] proposed TOGA,

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

a neural method for test oracle generation. TOGA is a unified transformer-based neural approach designed to infer both exceptional and assertion test oracles by leveraging the context of the focal method. In practice, developers often follow some common patterns to write unit tests. They begin by analyzing such patterns and outlining a taxonomy. To be more precise, they establish a straightforward grammar to express such taxonomy. Employing this grammar, coupled with type-based constraints, they confine the set of potential oracles, generating well-structured test oracles that adhere to both syntactic and type correctness criteria. Alonso et al. [64] proposed an automated approach (AGORA) to generate test oracles for REST APIs. AGORA learns invariants from API requests and returns instrumentation of the API requests consisting of a declaration file and a data trace file. Daikon [23] will process the corresponding results, and generate a set of likely invariants. If such invariants can be confirmed by developers, they can be used as test oracles.

Shahamiri et al. [8] proposed a technique to automatically generate test oracles by learning from the dynamic execution of the software under test (SUT) itself. The key rationale of this approach is that testers may train a machine-learning model to simulate SUT with sample input-output, and the resulting model could be leveraged as a test oracle to verify new tests. To this end, they leveraged a Multilayer Perceptron, a special category of neural networks, to simulate SUT's functionality. To train networks, they manually constructed training data: sample input-output of the SUT. Notably, the input could be generated automatically according to the domain of input variables whereas the expected output is constructed manually. Later, they further improved the approach by leveraging multiple neural networks to simulate more complex systems with multiple outputs [65].

Our approach is similar to such approaches in that all of them simulate the SUT with the input-output pairs of the SUT. However, our approach differs from theirs in that we do not require expected output. Furthermore, we focus on unit testing only, and simulate simpler methods instead of more complex software systems.

Motwani and Brun [66] presented Swami, a technique for automatically generating test oracles from structured natural language specifications. Swami employs regular expressions to extract testable behavior from the specifications and focuses on exceptional and boundary behavior that developers often miss. Their technique consists of three steps: identifying parts of the documentation relevant to the implementation to be tested, extracting test templates from those parts of the documentation, and generating executable tests from those templates. Evaluated on the JavaScript specification (ECMA-262), Swami-generated tests were found to be highly precise and effective in improving test coverage and identifying defects in JavaScript implementations.

Liu et al. [67] proposed ExLi, a technique that extracts inline tests from unit tests by first recording all variable values at a target statement during the execution of unit tests. ExLi then uses those values as test inputs and test oracles in an initial set of generated inline tests. To address redundancy, ExLi introduces a coverage-then-mutants based reduction process to remove redundant inline tests. Evaluated on 31 open-source programs, ExLi significantly reduces the number of initial inline tests while enhancing the fault-detection capability of the test suites.

As recent advancements have shown, LLMs are increasingly being applied to the field of test oracle generation. LLMs have demonstrated their potential in understanding and generating complex code and natural language constructs, making them a valuable tool in this domain. For instance, Mastropaolo et al. [68], [69] proposed a method for generating test oracles using a pre-trained T5 model. The model is first trained on a dataset composed of natural language text and source code, then fine-tuned on datasets used in previous studies for test assertion generation and program repair. This method achieved a 57% exact match rate for generated test assertions. Tufano et al. [70] also utilized a similar approach with separate pre-training on English and code corpora, followed by fine-tuning on a dataset containing test methods, focal methods, and assertions, achieving a 62% exact match rate. Nie et al. [71] extended this by incorporating six types of code semantics data, including execution results and context, to fine-tune the LLMs, improving their understanding of code execution information. Based on a different dataset, this method achieved a 17% exact match rate for test assertions. Nashid et al. [72] explored prompt engineering for test assertion generation, proposing a technique that automatically retrieves similar code demonstrations based on embedding or frequency analysis. This method, with a few relevant code demonstrations, achieved a 76% accuracy for exact matches in test assertion generation.

C. Detecting and Exploiting Invariants

Invariants play a crucial role in software testing and verification as they specify properties that hold at every execution of the SUT. By defining pre-conditions and post-conditions, invariants establish the expected behavior and constraints of the software. For example, for a method *sum(a, b)* that returns the sum (noted as c) of two positive integers a and b, the invariants $a > 0$ and $b > 0$ specify the pre-conditions of the method whereas $c = a + b$ and $c \geq 2$ specify the post-conditions of the method.

Both invariants and metamorphic relations specify properties of the SUT, and violating any of them could suggest illegal input or defects of the software. Invariants differ from metamorphic relations in that invariants specify properties that hold at a single run of the software whereas metamorphic relations specify the relations among multiple runs.

Daikon [23] is one of the most well-known approaches to detecting invariants of software. It instruments a set of predefined tests, runs the tests, and collects the logs (e.g., input and output of each public method). Based on the logs, Daikon validates a set of predefined potential invariants, and generates such invariants that hold on each run of the tests. Such invariants could be leveraged to design or validate tests. For example, Xie and Notkin [73] exploited the invariants generated by Daikon to select unit tests. Based on a set of verified tests, they leveraged Daikon to generate a set of invariants and verified automatically

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

generated unit tests against the resulting invariants. Tests violating the invariants were selected for manual verification.

D. Detecting Metamorphic Relations

Metamorphic relations are frequently leveraged to identify fault-revealing tests [74]. However, manual construction of such metamorphic relations could be tedious and time-consuming because the identification often involves much human intelligence for analyzing specifications and finding necessary properties of the SUT. Consequently, a few approaches have been proposed to derive metamorphic relations automatically. For example, Zhang et al. [75] proposed a search-based approach to inferring metamorphic relations automatically from Scratch based on the dynamic executions of a software system, and leveraged the resulting metamorphic relations to verify the following versions of the system. Notably, this approach is confined to a special category of metamorphic relations, i.e., polynomial metamorphic relations. Kanewala and Bieman [76] predicted whether given metamorphic relations hold for the SUT by a machine learning (SVM) based classifier. The input of the classifier is the control flow graph of the SUT, and the boolean output indicates whether a given metamorphic relation holds. Notably, this approach relies on labeled training data that are manually created in their evaluation. Recently, metamorphic testing has also been successfully applied to AI systems (models) [77].

VI. Conclusion and Future Work

It is easy to design a few unit tests to validate the major functionalities of a method. However, it remains challenging to conduct thorough unit testing automatically to identify hard-to-detect defects because of the well-known test oracle problem: We are lacking of test oracles to decide whether a test case fails or passes. To this end, in this paper, we propose a deep learning-based approach, called *OneMoreTest*, to generating and selecting fault-revealing unit tests. It selects the most suspicious tests by comparing their actual output against the estimated output generated by a deep learning model. Our evaluation results on 178 real-world faulty methods suggest that we may identify 70.79% defects by manually validating a single test for each of the faulty methods according to the suggestions of the proposed approach. The performance (70.79%) is much better than alternative approaches (46.63%). The major scenario in which developers might be interested in using *OneMoreTest* is when developers have tested the newly added/modified method with ad hoc test cases (accompanied by automated or manual test oracles) but still want more thorough testing. In this case, they have a great chance to identify the buggy behavior of the method, in case the method does contain bugs, by checking only one more test case. Notably, the major contribution of the approach is the ranking of suspicious test cases, and the purpose of the test case generation step is only to generate training data for the employed neural network that should be trained to select suspicious test cases. It is not intended to replace existing test case generation approaches. Consequently, the proposed approach is especially useful when developers have many test cases but do not have any explicit test oracles. In this case, they may leverage the proposed approach to select the most suspicious test cases for manual validation.

Our evaluation is conducted on Defects4J 1.0, which has been widely used in related research, to ensure comparability with prior studies. We have spent over three months extensively experimenting with this version, as it provides a well-established benchmark for evaluating testing techniques. The choice of Defects4J 1.0 allows us to compare our approach effectively with alternative methods and validate its performance under conditions commonly studied. Notably, the faulty methods and bugs we selected from Defects4J 1.0 are also included in the latest version. In future work, we are committed to extending our evaluation using the latest version of Defects4J and other datasets with more complex faults. This will enable us to further validate the scalability and robustness of our approach in more diverse testing scenarios. While our current evaluation focuses on real-world bugs from Defects4J, an important direction for future work is to extend the assessment to experiments using program mutants. The prototype implementation of the proposed approach is specially designed for Java because it leverages a Java parser and JUnit to identify input and output. However, the approach itself is not language-specific. In future, we plan to extend the prototype implementation of the approach to more programming languages. It is also interesting to evaluate the proposed approach with faulty methods in various programming languages.

References

[1] K. Beck, *Test-Driven Development: By Example.* Lebanon, IN, USA: Addison-Wesley Professional, 2003.

[2] D. Hamlet and R. Taylor, "Partition testing does not inspire confidence (program testing)," *IEEE Trans. Softw. Eng.*, vol. 16, no. 12, pp. 1402–1411, Dec. 1990.

[3] G. Fraser and A. Arcuri, "Whole test suite generation," *IEEE Trans. Softw. Eng.*, vol. 39, no. 2, pp. 276–291, Feb. 2013.

[4] G. Fraser and A. Arcuri, "EvoSuite: Automatic test suite generation for object-oriented software," in *Proc. 19th ACM SIGSOFT Symp. 13th Eur. Conf. Found. Softw. Eng. (ESEC/FSE)*. New York, NY, USA: ACM, 2011, pp. 416–419, doi: 10.1145/2025113.2025179.

[5] C. Pacheco and M. D. Ernst, "Randoop: Feedback-directed random testing for Java," in *Proc. Companion 22nd ACM SIGPLAN Conf. Object-Oriented Programm. Syst. Appl. Companion (OOPSLA)*, New York, NY, USA: ACM, 2007, pp. 815–816, doi: 10.1145/1297846.1297902.

[6] L. S. Pinto, S. Sinha, and A. Orso, "Understanding myths and realities of test-suite evolution," in *Proc. ACM SIGSOFT 20th Int. Symp. Found. Softw. Eng. (FSE)*, New York, NY, USA: ACM, 2012, doi: 10.1145/2393596.2393634.

[7] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Trans. Softw. Eng.*, vol. 41, no. 5, pp. 507–525, May 2015.

[8] S. R. Shahamiri, W. M. N. W. Kadir, S. Ibrahim, and S. Z. M. Hashim, "An automated framework for software test oracle," *Inf. Softw. Technol.*, vol. 53, no. 7, pp. 774–788, Jul. 2011, doi: 10.1016/j.infsof.2011.02.006.

[9] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for java programs," in *Proc. Int. Symp. Softw. Testing Anal. (ISSTA)*, New York, NY, USA: ACM, 2014, pp. 437–440, doi: 10.1145/2610384.2628055.

[10] "Replication Package", 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://github.com/OneMoreTest/OneMoreTest

[11] JavaParser, 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://javaparser.org/

[12] Oracle, "Oracle literals," 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://docs.oracle.com/cd/E19226-01/820-7627/6nisfjn2r/index.html

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

[13] OpenAI GPT, 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://openai.com/research/gpt

[14] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, Feb. 2012.

[15] CoverageDetails, 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://github.com/OneMoreTest/OneMoreTest/blob/main/Data/CoverageDetails.md

[16] I. Goodfellow, Y. Bengio, A. Courville, and Y. Bengio, *Deep Learning.* Cambridge, MA, USA: MIT Press, Nov. 2016.

[17] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Aug. 2009.

[18] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Trans. Pattern Anal. Mach. Intell*, vol. 35, no. 8, pp. 1798–1828, Aug. 2013, doi: 10.1109/TPAMI.2013.50.

[19] DeepAI, "Epoch definition," 2024. Accessed: Jul. 19, 2024. [Online]. Available: https://deepai.org/machine-learning-glossary-and-terms/epoch

[20] Z. Xu, X. Chang, F. Xu, and H. Zhang, "L-1/2 regularization: A thresholding representation theory and a fast solver," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 23, no. 7, pp. 1013–1027, Jul. 2012.

[21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.

[22] E. S. Ristad and P. N. Yianilos, "Learning string-edit distance," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 5, pp. 522–532, May 1998.

[23] M. D. Ernst, J. Cockrell, W. G. Griswold, and D. Notkin, "Dynamically discovering likely program invariants to support program evolution," *IEEE Trans. Softw. Eng.*, vol. 27, no. 2, pp. 99–123, Feb. 2001.

[24] F. Li, J. Zhou, Y. Li, D. Hao, and L. Zhang, "Aga: An accelerated greedy additional algorithm for test case prioritization," *IEEE Trans. Softw. Eng.*, vol. 48, no. 12, pp. 5102–5119, Dec. 2022.

[25] B. Jiang, Z. Zhang, W. K. Chan, and T. H. Tse, "Adaptive random test case prioritization," in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2009, pp. 233–244.

[26] H. Hemmati, L. Briand, A. Arcuri, and S. Ali, "An enhanced test case selection approach for model-based testing: An industrial case study," in *Proc. 18th ACM SIGSOFT Int. Symp. Found. Softw. Eng. (FSE)*, New York, NY, USA: ACM, 2010, pp. 267–276, doi: 10.1145/1882291.1882331.

[27] D. Huang et al., "Neuron sensitivity guided test case selection," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 7, pp. 1–32, Jun. 2024, doi: 10.1145/3672454.

[28] J. Campos, Y. Ge, N. Albunian, G. Fraser, M. Eler, and A. Arcuri, "An empirical evaluation of evolutionary algorithms for unit test suite generation," *Inf. Softw. Technol.*, vol. 104, pp. 207–235, Dec. 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584917304858

[29] "Selected Real-world Faulty Methods," 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://github.com/OneMoreTest/OneMoreTest/blob/main/Data/SelectedReal-worldFaultyMethods.md

[30] "Replication of OneMoreTest Experimental Data," 2024, doi: 10.5281/zenodo.15678975.

[31] "Replication of baseline experimental data," 2024, doi:10.5281/zenodo.13295030.

[32] "Evaluation results," 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://github.com/OneMoreTest/OneMoreTest/blob/main/Data/EvaluationResults.md

[33] "DateUtils.java," 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://github.com/apache/commons-lang/blob/master/src/main/java/org/apache/commons/lang3/time/DateUtils.java

[34] "Complex.java," 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://github.com/haidaros/defects4j-math/blob/master/src/main/java/org/apache/commons/math3/complex/Complex.java

[35] "LevenbergMarquardtOptimizer.java," 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://github.com/haidaros/defects4j-math/blob/master/src/main/java/org/apache/commons/math3/optimization/general/LevenbergMarquardtOptimizer.java

[36] K. Levenberg, "A method for the solution of certain non-linear problems in least squares," *Q. Appl. Math.*, vol. 2, no. 2, pp. 164–168, 1944. [Online]. Available: http://www.jstor.org/stable/43633451

[37] M. D. Ernst et al., "The daikon system for dynamic detection of likely invariants," *Sci. Comput. Program.*, vol. 69, nos. 1–3, pp. 35–45, 2007, doi: 10.1016/j.scico.2007.01.015.

[38] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2019, pp. 830–840, doi: 10.1145/3338906.3338945

[39] X. Chen et al., "Deep learning-based software engineering: progress, challenges, and opportunities," *Sci. China Inf. Sci.*, vol. 68, no. 1, pp. 1–88, 2025.

[40] Google AutoML, 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://cloud.google.com/automl/

[41] Auto-Keras, 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://autokeras.com/

[42] OpenAI Codex, 2024. Accessed: Jun. 26, 2025. [Online]. Available: https://openai.com/research/codex

[43] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.

[44] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020. [Online]. Available: http://jmlr.org/papers/v21/20-074.html

[45] H. Touvron et al., "Llama: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[46] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, Apr. 2024.

[47] H. Zhu, P. A. V. Hall, and J. H. R. May, "Software unit test coverage and adequacy," *ACM Comput. Surv.*, vol. 29, no. 4, pp. 366–427, Dec. 1997, doi: 10.1145/267580.267590.

[48] P. Ammann and J. Offutt, *Introduction to Software Testing*, 2nd ed. Cambridge, MA, USA: Cambridge Univ. Press, 2016.

[49] L. Inozemtseva and R. Holmes, "Coverage is not strongly correlated with test suite effectiveness," in *Proc. 36th Int. Conf. Softw. Eng. (ICSE)*, 2014, New York, NY, USA: ACM, 2014, pp. 435–445, doi: 10.1145/2568225.2568271.

[50] R. Kazmi, D. N. A. Jawawi, R. Mohamad, and I. Ghani, "Effective regression test case selection: A systematic literature review," *ACM Comput. Surv.*, vol. 50, no. 2, pp. 1–32, May 2017, doi: 10.1145/3057269.

[51] M. J. Harrold et al., "Regression test selection for Java software," *SIGPLAN Notes*, vol. 36, no. 11, pp. 312–326, Oct. 2001, doi: 10.1145/504311.504305.

[52] Y. Liu, J. Zhang, P. Nie, M. Gligoric, and O. Legunsen, "More precise regression test selection via reasoning about semantics-modifying changes," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, New York, NY, USA: ACM, 2023, pp. 664–676, doi: 10.1145/3597926.3598086.

[53] A. Arrieta, P. Valle, J. A. Agirre, and G. Sagardui, "Some seeds are strong: Seeding strategies for search-based test case selection," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 1, Feb. 2023, doi: 10.1145/3532182.

[54] M. Khatibsyarbini, M. A. Isa, D. N. Jawawi, and R. Tumeng, "Test case prioritization approaches in regression testing: A systematic literature review," *Inf. Softw. Technol.*, vol. 93, pp. 74–93, 2018.

[55] S. Mondal and R. Nasre, "Colosseum: Regression test prioritization by delta displacement in test coverage," *IEEE Trans. Softw. Eng.*, vol. 48, no. 10, pp. 4060–4073, Oct. 2022.

[56] S. Wang, J. Nam, and L. Tan, "QTEP: Quality-aware test case prioritization," in *Proc. 11th Joint Meeting Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2017, pp. 523–534, doi: 10.1145/3106237.3106258.

[57] A. Ramírez, R. Feldt, and J. R. Romero, "A taxonomy of information attributes for test case prioritisation: Applicability, machine learning," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 1, Feb. 2023, doi: 10.1145/3511805.

[58] J. A. P. Lima and S. R. Vergilio, "A multi-armed bandit approach for test case prioritization in continuous integration environments," *IEEE Trans. Softw. Eng.*, vol. 48, no. 2, pp. 453–465, Feb. 2022.

[59] A. S. Yaraghi, M. Bagherzadeh, N. Kahani, and L. C. Briand, "Scalable and accurate test case prioritization in continuous integration contexts," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 1615–1639, Apr. 2023.

[60] M. Bagherzadeh, N. Kahani, and L. Briand, "Reinforcement learning for test case prioritization," *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 2836–2856, Aug. 2022.

[61] E. Engström, P. Runeson, and M. Skoglund, "A systematic review on regression test selection techniques," *Inf. Softw. Technol.*, vol. 52, no.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.

1, pp. 14–30, 2010. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584909001219

[62] T. L. Graves, M. J. Harrold, J.-M. Kim, A. Porter, and G. Rothermel, "An empirical study of regression test selection techniques," *ACM Trans. Softw. Eng. Methodol*, vol. 10, no. 2, pp. 184–208, Apr. 2001, doi: 10.1145/367008.367020.

[63] E. Dinella, G. Ryan, T. Mytkowicz, and S. K. Lahiri, "Toga: A neural method for test oracle generation," in *Proc. 44th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2022, pp. 2130–2141, doi: 10.1145/3510003.3510141.

[64] J. C. Alonso, S. Segura, and A. Ruiz-Cortés, "Agora: Automated" generation of test oracles for rest APIs," in *Proc 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal., (ISSTA)*, New York, NY, USA: ACM, 2023, pp. 1018–1030, doi: 10.1145/3597926.3598114.

[65] S. R. Shahamiri, W. M. Wan-Kadir, S. Ibrahim, and S. Z. Hashim, "Artificial neural networks as multi-networks automated test oracle," *Automated Softw. Eng.*, vol. 19, no. 3, pp. 303–334, Sep. 2012, doi: 10.1007/s10515-011-0094-z.

[66] M. Motwani and Y. Brun, "Automatically generating precise oracles from structured natural language specifications," in *Proc. 41st Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 188–199, doi: 10.1109/ICSE.2019.00035.

[67] Y. Liu, P. Nie, A. Guo, M. Gligoric, and O. Legunsen, "Extracting inline tests from unit tests," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal. (ISSTA)*, New York, NY, USA: ACM, 2023, pp. 1458–1470, doi: 10.1145/3597926.3598149.

[68] A. Mastropaolo et al., "Using transfer learning for code-related tasks," *IEEE Trans. Softw. Eng*, vol. 49, no. 4, pp. 1580–1598, Apr. 2023, doi: 10.1109/TSE.2022.3183297.

[69] A. Mastropaolo et al., "Studying the usage of text-to-text transfer transformer to support code-related tasks," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE),* 2021, pp. 336–347.

[70] M. Tufano, D. Drain, A. Svyatkovskiy, and N. Sundaresan, "Generating accurate assert statements for unit test cases using pre-trained transformers," in *Proc. 3rd ACM/IEEE Int. Conf. Automat. Softw. Test (AST)*, New York, NY, USA: ACM, 2022, pp. 54–64, doi: 10.1145/3524481.3527220.

[71] P. Nie, R. Banerjee, J. J. Li, R. J. Mooney, and M. Gligoric, "Learning deep semantics for test completion," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 2111–2123, doi: 10.1109/ICSE48619.2023.00178.

[72] N. Nashid, M. Sintaha, and A. Mesbah, "Retrieval-based prompt selection for code-related few-shot learning," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 2450–2462, doi: 10.1109/ICSE48619.2023.00205.

[73] T. Xie and D. Notkin, "Tool-assisted unit test selection based on operational violations," in *Proc. 18th IEEE Int. Conf. Automated Softw. Eng.,* 2003, pp. 40–48.

[74] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Trans. Softw. Eng.*, vol. 42, no. 9, pp. 805–824, Sep. 2016.

[75] J. Zhang, et al., "Search-based inference of polynomial metamorphic relations," in *Proc. 29th ACM/IEEE Int. Conf. Automat. Softw. Eng. (ASE)*, New York, NY, USA: ACM, 2014, pp. 701–712, doi: 10.1145/2642937.2642994.

[76] U. Kanewala and J. M. Bieman, "Using machine learning techniques to detect metamorphic relations for programs without test oracles," in *Proc. IEEE 24th Int. Symp. Softw. Rel. Eng. (ISSRE),* Nov. 2013, pp. 1–10.

[77] X. Xie, P. Yin, and S. Chen, "Boosting the revealing of detected violations in deep learning testing: A diversity-guided method," in *Proc. 37th IEEE/ACM Int. Conf. Automat. Softw. Eng. (ASE)*, New York, NY, USA: ACM, 2023, doi: 10.1145/3551349.3556919.

Wei Wei received the B.S. degree in software engineering from the Central South University, Changsha, in 2022. She is currently working toward the master's degree with the School of Computer Science and Technology, Beijing Institute of Technology. Her research focuses on intelligent software engineering, with particular interests in software testing, automated test generation, and the integration of large language models into testing workflows.

Yanjie Jiang received the Ph.D. degree in computer science from Beijing Institute of Technology, in 2023. She is an Associate Professor with the College of Intelligence and Computing, Tianjin University, China. She was a Postdoctoral Researcher with Peking University. She served on the Program Committees of many prestigious conferences, such as FSE, ICSE, and ASE. Her current research interests include software testing and analysis, software maintenance, and evolution.

Yahui Li received the B.S. degree from Xidian University, Xi'an, and the M.S. degree from Beijing Institute of Technology, both in computer science and technology. His research interests include deep learning and software testing.

Lu Zhang received the B.Sc. and Ph.D. degrees in computer science from Peking University in 1995 and 2000, respectively. He is a Professor with the School of Electronics Engineering and Computer Science, Peking University, China. He was a Postdoctoral Researcher with Oxford Brookes University and University of Liverpool, U.K. He served on the Program Committees of many prestigious conferences, such as FSE, OOPSLA, ISSTA, and ASE. He was a Program Co-Chair of SCAM 2008 and ICSM 2017. He has served on editorial boards of journals including software testing, verification, and reliability. His current research interests include software testing and analysis, program comprehension, software maintenance and evolution, software reuse, and program synthesis.

Hui Liu is currently a Professor with the School of Computer Science and Technology, Beijing Institute of Technology, China. He serves as the Secretary General of the CCF Technical Committee on software engineering, a PC Co-Chair of ChinaSoft 2025–2026, and an Associate Editor of IET Software. His research interests include software refactoring, AI-based software engineering, and software quality, as well as practical tools to assist software engineers.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on January 11,2026 at 11:41:55 UTC from IEEE Xplore. Restrictions apply.